PROJECT MANUAL / EDITION 1.0

Folio

A reading room for the digital age.

Structural and design manual

Product scope, interface design, system architecture, data, APIs, intelligent assistance, music, discovery and community.

REACT + TYPESCRIPT

NODE.JS + EXPRESS / MONGODB

EPUB-FIRST / PRIVATE BY DEFAULT / BUILT IN PHASES

Read naturally. Understand deeply. Imagine vividly.

23 October 2026

Working name and proposed specification. Screen data is illustrative. No implementation or measured performance is claimed.

Contents

1. Product definition and boundaries 4

2. Feature map and release boundaries 6

3. Information architecture and user journeys 8

4. Visual identity and design system 10

5. Screen-by-screen interface specification 13

6. System architecture and service responsibilities 24

7. Repository structure and engineering conventions 27

8. Data model and ownership 28

9. HTTP API contracts 30

10. Reading progress, daily statistics and offline consistency 33

11. Contextual AI, translation and scene generation 35

12. Music and adaptive reading audio 37

13. Personalized book discovery 38

14. Friends, privacy and challenges 39

15. Security, privacy and content handling 40

16. Performance, accessibility and resilience 42

17. Verification, operations and cost control 44

18. Implementation sequence and completion gates 46

19. Acceptance checklist and open decisions 48

20. Detailed workflows and handoff contracts 50

21. Primary technical references 53

How to use this manual

Use Chapters 1-5 to align the product owner and designer, Chapters 6-14 to guide implementation, and Chapters 15-19 to plan security, operations, validation, and delivery. Start with the release matrix; the complete product should emerge through several working releases. Proposed defaults can change after user testing without changing the central reading experience.

The manual preserves the original brief, including the later additions for music, history-based recommendations, daily streaks, average pages per day, friends, and challenges. The original attached brief supplied the detailed reader and design requirements. Some previous assistant messages were available only as reference markers; no missing decision is assumed to be confirmed.

Screen blueprints illustrate layout and visual hierarchy. Their sample books, activity, and statistics are demonstration content. Runtime performance figures are engineering targets to validate, not measured results. The final chapter lists primary technical references, checked on the edition date.

1. Product definition and boundaries

1.1 Purpose and audience

Folio is a private digital library and reading environment with contextual assistance. It helps readers understand language, interpret a passage, remember an ongoing book, and build a sustainable reading habit. The text remains the primary interface. Intelligence appears when requested and returns the reader to the same position afterward.

The first audience includes readers working in a second language, fiction readers who want interpretation without spoilers, people returning to a book after a gap, and readers who want quiet accountability with friends. Students and non-fiction readers also benefit from linked explanations, vocabulary, and notes. Avoid promising academic or factual correctness solely because an answer was generated.

1.2 Distinctive product value

- Contextual understanding: explain a word or passage using the book context that the reader has permitted.
- Reading continuity: synchronize position, notes, preferences, and a spoiler-safe recap.
- Optional immersion: visualize a scene and select suitable reading music without interrupting navigation.
- Meaningful discovery: recommend books using explicit preferences and reading history, and explain the match.
- Gentle accountability: show understandable reading statistics and optional social challenges.

The product succeeds when reading is comfortable, explanations are useful, and users feel in control. Session length, AI requests, or notification clicks alone are weak success measures.

1.3 Product guardrails

Keep book text prominent. AI, music, and social features must be independently optional. A failed AI or music request must leave the reader usable. Do not silently autoplay audio, share a private upload, publish reading activity, or submit personal notes to a model. Do not infer that moving to the end of a book means the user read it.

Support user-authorized material and a permitted sample catalog. Store book metadata separately from a user's private file. A public metadata result does not grant the right to distribute the corresponding book. Provider and content rights need review for the actual launch regions; this is an implementation boundary rather than a statement of legal advice.

1.4 Roles

Role	Capabilities	Boundary
Visitor	Read product information and a permitted sample; explore the interface	No private library or persistent AI history
Reader	Manage own library, read, annotate, use enabled assistance, manage friends	Access only own private resources and explicitly shared social data
Community moderator	Review reported discussions and take scoped moderation actions	No routine access to private book files, notes, or AI conversations
Operator	Manage jobs, quotas, service health, and support processes	Least-privilege tools; sensitive access requires an audited workflow

1.5 Measures of usefulness

Track successful book imports, successful returns to a saved location, active reading days, explanation helpfulness, corrected recommendations, and accessible task completion. Use aggregate, consent-aware telemetry. A reader who disables streaks or social features is still a successful user. Survey why people abandon the reader before adding new engagement mechanics.

2. Feature map and release boundaries

2.1 Release matrix

Capability	Release	First complete behavior	Expansion
Account and library	R1	Secure sign-in, private EPUB upload, import status, metadata editing	Multiple formats and account export
Reader	R1	EPUB chapters, themes, keyboard navigation, saved position	PDF/TXT, broader layout support, offline downloads
Contextual translation	R1	Selection and paragraph translation with target-language control	Dictionary audio, richer bilingual display
Explanation	R1	Simple/deep explanation grounded in selected passage and allowed context	Discussion partner and advanced literary analysis
Annotations	R1	Bookmarks, highlights, notes, stable anchors	Journal export and richer search
Reading activity	R1	Reliable event collection and private basic totals	Insights, streaks, challenges
Spoiler boundaries	R1	Applied to all R1 AI requests	Same policy for every later AI and social feature
Scene images	R2	Explicit generation, job status, personal gallery	Character continuity and multiple styles
Story Memory / Character Compass	R2	Recap and character facts from allowed context	Relationship diagrams and richer references
Vocabulary	R2	Save word with passage and translation; optional review	Spaced repetition and quizzes
Music	R2	User-started licensed instrumental/ambient playback and suggestions	Optional adaptive transitions and provider integrations
Recommendations	R2	Explicit tastes plus eligible history, with reasons and feedback	Better ranking with sufficient consented data
Insights	R2	Current/longest streak, average pages, goals and date filters	Monthly/yearly reflections
Friends and challenges	R3	Requests, blocking, opt-in activity, private challenges	Reading circles and spoiler-controlled discussions
Multi-device offline	R3	Explicit downloads, queued progress/notes, reconciliation	Conflict improvements after field testing

R1 is the reading MVP. Recording trustworthy activity and enforcing spoiler controls start there; displaying sophisticated analytics and social comparisons can wait. Do not enable a feature publicly until its error states, permission rules, and acceptance criteria work.

2.2 Complete product modules

1. Identity and settings: sessions, language, timezone, accessibility, privacy, notifications, retention, account deletion.

2. Library and ingestion: uploads, permitted sample catalog, import jobs, document versions, formats and file lifecycle.
3. Reader and annotations: rendering, canonical locations, progress, highlights, notes, search, bookmarks and preferences.
4. Reading intelligence: translation, interpretation, recap, character facts, vocabulary and image generation.
5. Audio: soundtrack catalog, rights metadata, user playback, preferences and optional adaptation.
6. Discovery: catalog metadata, feedback, taste profile, ranked suggestions and explanations.
7. Habits and community: statistics, goals, friend graph, challenges, circles, discussions, notifications and moderation.
8. Platform: jobs, storage, observability, access control, quotas, exports, administration and recovery.

2.3 Decisions to validate before public launch

Confirm the product name; launch languages and regions; content and soundtrack sources; AI and storage vendors; deployment budgets; and account eligibility. Defaults in this manual are EPUB-first, private profiles, user-started audio, English navigation with localization support, and original-language reading with selectable translations. Urdu and Arabic content require RTL handling from the start even if the first navigation release is English.

3. Information architecture and user journeys

3.1 Navigation and pages

Route	Page	Primary purpose
`/`	Landing	Show the reading experience, explain value, offer a permitted sample
`/sample`	Sample reader	Let a visitor read and try a constrained demonstration
`/auth/sign-in`, `/auth/register`	Account	Enter a session; explain required terms and privacy choices
`/library`	Personal library	Continue a book, import content, browse shelves
`/library/:entryId`	Library entry	Book details, status, notes summary, format, removal
`/read/:entryId`	Reader	Render text and contextual tools
`/discover`	Discover	Browse recommended and curated book metadata
`/books/:catalogId`	Public book details	Metadata, recommendation reason, permitted acquisition link
`/journal`	Journal	Search highlights, notes, reflections and vocabulary
`/insights`	Reading insights	Understand activity, streaks, goals and average pages
`/friends`	Friends	Requests, accepted friends, blocking and activity controls
`/challenges`, `/challenges/:id`	Challenges	Create, join and track explicit shared goals
`/circles/:id`	Reading circle	Opt-in, progress-controlled discussion in a later release
`/settings/*`	Settings	Reading, language, audio, privacy, notifications, account
`/admin/*`	Operator tools	Authorized operational actions on a separately restricted interface

Library, Discover, Journal, Insights, and Friends form primary navigation. The reader uses a reduced navigation shell with a clear return to the originating screen, with Library as a fallback. Settings live under the account menu. Do not display unavailable release routes as functional features.

3.2 First visit to first page

Show an editorial landing page with a real sample reader preview. A visitor can try a permitted sample without completing an account flow. On registration ask only for required identity information. Collect target language and reading theme in a small optional setup step; taste preferences can wait until Discover.

The empty library offers two useful actions: upload an EPUB or open a sample. On upload, show the filename, progress, processing state and supported limit before the user commits. A successful import opens book details or a clear Start reading action. Preserve accessible error details for unsupported, encrypted, corrupt, or oversized files.

3.3 Reading and understanding a passage

The user opens a book at the last synchronized location. Selecting text reveals a compact toolbar. Translate uses the saved target language and allows a quick change. Explain opens a compact side panel on desktop or an accessible sheet on mobile. The panel states the selected passage, explanation mode and contextual boundary; it never displaces the reading position.

A reader can save a useful word or attach a note. Successful saves receive quiet confirmation. Closing the panel returns focus to the selection. No selection should disappear because an unrelated component rerendered. The reader remains navigable while a request is in progress.

3.4 Returning after a break

The library puts Continue reading near the top. Show title, chapter and progress using the reader's actual data. Offer a short Previously in your book recap on request; generate it against the current explicit spoiler boundary. A jump to a later location should ask whether the permitted context should expand before including that text in AI outputs.

3.5 Music, discovery and friendship

Music begins from an explicit Play action. Offer a simple soundtrack choice and volume; adaptation remains a separate preference. The user can lock a soundtrack, pause, or disable audio for the book.

Discover explains why each book appears and offers Not interested and Exclude this author/genre controls. A friend invitation exposes a display name and chosen profile information, never a private upload. Joining a challenge shows the exact metric, dates, timezone and visibility before confirmation.

3.6 Destructive and sensitive actions

Book removal distinguishes removing a library entry from deleting the associated private file and annotations. Account deletion explains the retention/deletion workflow and confirms before submission. Exports should be available through an authenticated job. Changing social visibility shows a preview of the fields that will become visible.

4. Visual identity and design system

This section describes the intended product design. It is a build specification, not a statement that any screen, integration, or feature already exists. **Folio** is a tentative working name; confirm naming and brand availability before a public release.

4.1 Product identity and design principles

Folio should feel like a quiet personal reading room: warm paper, expressive editorial typography, precise tools, and restrained animation. Its promise is "Read with understanding, keep your rhythm, discover what comes next." The core experience is the book itself. AI, music, progress, and community should remain available without becoming competing feeds.

Use these principles to resolve design decisions:

1. **Protect the passage.** Keep reading text central. Open supporting content beside the text on wide screens and in dismissible sheets on smaller screens. Preserve the reader's location after every action.
2. **Make assistance intentional.** Trigger translation, explanation, and illustration from a reader action. Explain the scope of information being sent before the first external processing request.
3. **Make progress encouraging.** Present streaks and page statistics as useful observations. Avoid guilt messages, flashing reminders, and public competition by default.
4. **Keep provenance visible.** Distinguish the original book, user annotations, machine translation, generated explanation, and generated image. A generated illustration is an interpretation, not evidence about the text.
5. **Keep the interface predictable.** Reuse the same spacing, controls, focus behavior, and panel arrangement. Offer the same core functionality through touch, keyboard, and assistive technology.

Use an understated wordmark and a simple open-page symbol. Book covers supply most of the color in the application. Reserve the primary accent for meaningful actions, selected states, and progress. Do not use it to decorate every card.

4.2 Color system and themes

Implement semantic CSS variables rather than hard-coded component colors. Application chrome can follow the operating-system theme initially; the reader has its own persistent Light, Sepia, and Dark choice. A reader preference must not unexpectedly recolor the rest of the application.

Semantic token	Light	Sepia reader	Dark
`--bg`	`#F8F6F1`	`#F1E7D3`	`#121B18`
`--surface`	`#FFFFFF`	`#F8EFDD`	`#1B2822`
`--surface-raised`	`#F0EEE7`	`#E8DCC2`	`#24362D`
`--text`	`#202A25`	`#382F26`	`#EDF2EB`
`--text-muted`	`#56635B`	`#655644`	`#B7C6BB`
`--accent`	`#2D5B4B`	`#435D44`	`#A7CDB8`
`--on-accent`	`#FFFFFF`	`#FFFFFF`	`#12241A`
`--border`	`#D8DDD5`	`#CCBDA3`	`#40564A`

Semantic token	Light	Sepia reader	Dark
`--control-border`	`#77867A`	`#8C7B63`	`#799582`
`--focus`	`#2458A6`	`#2458A6`	`#9FC4FF`
`--danger-text`	`#A12727`	`#8C2525`	`#FFB4AB`
`--success-text`	`#256244`	`#315D35`	`#A5D6B0`
`--warning-text`	`#795000`	`#704500`	`#EBC776`

Use `--border` for decorative separators only. Required field outlines, unchecked controls, and other boundaries necessary to identify an interactive element use `--control-border`. Do not assume that every possible combination of these tokens is accessible. Validate the actual rendered foreground/background pairs, including hover, disabled, chart, and overlay states. Target at least 4.5:1 contrast for normal text, 3:1 for large text, and 3:1 for meaningful graphics and control boundaries. Color must never be the sole indicator of an error, selection, streak, or challenge result.

Annotations can offer yellow `#F4D96B`, mint `#B8DCC4`, blue `#B8D5F3`, and pink `#EBC0CE`. Treat these as translucent backgrounds with the reader's normal text color and test the composited result in each theme. Underline or outline the active annotation as well. Respect forced-colors mode; do not force custom backgrounds when doing so hides system focus or selection.

4.3 Typography, layout, and motion

Use **Literata** as the proposed Latin reading face and **Inter** for interface text, with self-hosted, appropriately licensed font files and system fallbacks. The reading font must remain user-selectable; include a system sans option. For Arabic-script content, propose **Noto Naskh Arabic** for passages and **Noto Sans Arabic** for controls. If Urdu receives a dedicated Nastaliq option, test its taller line boxes and clipping separately. Confirm language coverage with real passages before adopting any font.

Interface type scale: 12 px metadata, 14 px secondary controls, 16 px body and primary controls, 20 px card titles, 28 px page headings, and 36 px occasional editorial headings. Do not place essential instructions at 12 px. Reader defaults: 20 px type, 1.7 line height, approximately 64 characters per line, left alignment for left-to-right languages, and start alignment for all language directions. Allow 16-32 px reader text, line heights of 1.4-2.1, and narrow/comfortable/wide text measures. Avoid full justification as the default.

Build on a 4 px spacing unit with preferred values of 4, 8, 12, 16, 24, 32, 48, and 64 px. Use 10 px control radii, 16 px card radii, and 20 px sheet radii. Cards receive a one-pixel border and little or no shadow. Only floating menus and dialogs receive a soft elevation shadow.

Use content-based responsive behavior with initial breakpoints at 640, 1024, and 1440 px. Under 640 px, use a single column and 16 px side gutters. From 640-1023 px, use 24 px gutters and two-column collections where practical. At 1024 px and above, provide a 232 px navigation rail, 32 px content gutters, and a 1280 px maximum application content width. At 1440 px and above, an optional supporting panel can appear without increasing the reading measure. Verify layout at intermediate widths rather than treating these numbers as device categories.

Keep most transitions at 120-180 ms, using opacity and small translations. Avoid animated page flips in the initial release. Honor reduced motion by eliminating positional animation and disabling decorative effects. Never move a reader's text because a statistic, notification, or generated result arrived.

4.4 Components, accessibility, and delivery acceptance

Define reusable buttons, inputs, menus, dialogs, sheets, tabs, book cards, annotation cards, metric cards, player controls, and status banners. For each, specify default, hover, keyboard focus, pressed/selected, loading, disabled, error, empty, and success states where relevant. A loading button keeps its label and width. Long operations expose meaningful status. Disable only the dependent action, preserving navigation and cancellation.

Use semantic headings, landmarks, links, buttons, lists, and form labels. Target WCAG 2.2 AA as the design acceptance standard. Keep touch targets comfortably sized, preferably 44 x 44 CSS px; provide visible focus and logical tab order. Dialogs manage initial focus, contain focus while modal, close by an explicit control and Escape where appropriate, and restore focus to their trigger. Announce asynchronous statuses politely, without reading the entire book again. Errors explain the correction and remain next to the relevant field.

At 200% text zoom and narrow layouts, all controls and passage text must remain usable without clipping. Test keyboard-only operation, a screen reader, dark/sepia/forced-colors modes, reduced motion, long translated labels, missing covers, offline mode, slow generation, and sync failures. Apply `dir` at passage and content boundaries, use logical CSS properties, isolate mixed-direction titles/numbers, and mirror directional controls where appropriate. Do not reverse universally recognized media controls solely because the page is RTL.

Before implementation, create a design file containing these tokens, component variants, the six primary screens at mobile and desktop widths, the reader assistance states, and representative empty/error/loading screens. The handoff is complete only when developers can trace every screen element to a reusable component or an explicitly documented exception.

5. Screen-by-screen interface specification

5.1 Information architecture and shared shell

The six principal destinations are **Library**, **Discover**, **Journal**, **Insights**, **Friends**, and the contextual **Reader**. Library, Discover, Journal, Insights, and Friends appear in persistent application navigation; Reader opens from a book and receives a distraction-free shell. Keep account, theme, privacy, language, and accessibility preferences in one Settings destination.

Show navigation only for available release features. On mobile, use a bottom bar for Library, Discover, Journal, Insights, and Friends with both icons and labels. Hide it inside the reader, where the back control returns to the originating screen, with Library as the fallback. Each page has a clear heading, a single primary action where needed, and contextual secondary actions. Search must show its scope: "Search your library," "Find books," or "Search your notes."

The global "Continue reading" affordance opens the most recent saved location and shows the book title. Never silently substitute another edition or reset to the first chapter.

5.2 Library

Purpose: find a book and resume reading with minimal effort. The header contains "Your library," a scoped search field, and **Add book**. Below it, show a prominent Continue Reading card with cover, title, author, chapter/location, progress, and **Resume**. Add a compact reading goal summary without making it larger than the book.

Provide All, Reading, Want to read, and Finished filters, followed by sorting and grid/list controls. Desktop cards show a consistent 2:3 cover, title, author, status, progress, and an accessible overflow menu. Missing covers use a restrained typographic placeholder. The list view emphasizes metadata and supports long titles. A new user sees an empty state explaining upload and discovery, with one primary action.

Add-book flow: choose file, validate format and size, show upload and processing progress, review title/author/cover, then **Add to library**. Separate "Uploading," "Preparing chapters," "Ready," and "Could not process" states. An unreadable document must retain a useful error and retry/remove actions. Do not promise text extraction for every PDF; a scanned document may require a separately designed OCR step. Deleting a book distinguishes removing the library entry, deleting the private file, and deleting annotations, with the consequence of each choice stated clearly.

5.3 Reader

Purpose: comfortable reading with contextual help. On desktop, use a compact top bar containing Back, abbreviated title, table of contents, search, appearance, and music. Center the passage in a configurable text column. The footer contains chapter/location, progress, and next/previous controls appropriate to the reading mode. Keep a stable source location behind visual pagination; font changes must not reset progress or inflate page statistics.

Text selection opens a compact menu with **Translate**, **Explain**, **Highlight**, **Note**, and **Illustrate scene** where applicable. These actions must also be reachable through keyboard-accessible passage/annotation controls; do not depend exclusively on a transient mouse-selection menu. A wide-screen assistance panel is approximately 360 px with a minimum usable passage width. At smaller widths, show an accessible sheet that can expand to full screen. The panel includes the selected passage, action label, language or explanation depth, result, and Close/Retry controls.

Translation should expose original and translated text together. Explanation shows the explicit scope, for example "Context through Chapter 3," with a Change boundary control. The saved reading location and allowed AI context are distinct. Scene images remain outside the passage unless the reader deliberately expands one. Label them "AI-generated interpretation," allow dismissal, and provide a useful text description. Generation has queued/loading/cancellation-requested/cancelled/error states; show Cancelled only after cancellation is confirmed; preserve the passage if the service fails.

Appearance controls include theme, font, type size, line height, width, and reading mode where supported. Bookmarks and notes use stable locations, not a screenshot coordinate. A small "Progress saved" status can appear without stealing focus. Surface offline and sync-conflict states honestly. On mobile, keep core controls available through an obvious toolbar toggle; touching ordinary text must not unexpectedly open several overlays.

5.4 Discover

Purpose: choose a next book. Begin with "Recommended for you," followed by a concise reason such as "Because you finished..." or "More historical fiction." Distinguish personalized results from editorial, popular, or starter suggestions. When history is insufficient, offer genre/language interests and a browsable catalog.

Cards include cover, title, author, language, short description, recommendation reason, and **View book**. A detail page shows synopsis, genre, length where available, source/availability, **Add to Want to read**, and **Start reading** only when readable content is actually available. Recommendation metadata alone does not imply full-book access.

Provide **More like this**, **Not interested**, and an explanation of how history influences suggestions. Filters include language, genre, and availability. Avoid revealing later plot events through recommendation explanations. A failed recommendation service should leave catalog search and saved lists usable.

5.5 Journal

Purpose: retrieve and develop personal observations. Show Notes and Highlights as tabs, add Vocabulary in R2, and offer Saved explanations as a tab or content filter, with search and book/date/tag filters. Each item displays a passage excerpt, the reader's own note where present, book title, chapter/location, and modified date. **Open in book** returns to the source location.

Use a two-pane list/detail arrangement on desktop and list-to-detail navigation on mobile. Preserve drafts when changing filters or closing a sheet. Show saved/saving/error states for edits, and offer recovery for failed saves. Distinguish editable personal text from a quoted passage and generated content. Notes are private by default; explicit sharing should show exactly which text and book metadata will become visible. Export options must describe the selected scope.

5.6 Insights

Purpose: understand reading habits without misleading precision. Start with current streak, longest streak, pages read in the chosen period, average pages per day, reading time, and completed books. Give every metric a short definition through an adjacent help control.

Offer 7-day, 30-day, and custom ranges. The average label must state whether it includes all calendar days or only active days; use all days as the proposed default. "Pages" for reflowable books require a documented normalized unit. Show "estimated pages" where source pages are unavailable, and do not add multiple visits to the same passage as newly completed pages. Streak day boundaries use the user's selected timezone and should remain predictable during travel.

Use a daily activity chart, a monthly calendar, and optional genre/language summaries. Provide a readable table alternative and accessible chart labels. Empty days are zero only when tracking data is available; unavailable data needs a distinct state. Set-goal controls should allow modest daily targets and easy adjustment. Avoid shaming copy when a streak ends.

5.7 Friends and challenges

Purpose: encourage reading with people the user chooses. Use Friends, Requests, and Challenges tabs. Support a searchable handle or deliberate invitation link, with clear request pending/accepted/declined states. Never expose email addresses in public search results.

Each friend card shows permitted profile information and explicit actions. Reading activity, current book, notes, and statistics need independent sharing controls; default profiles and reading history to private. Show an audience preview

before posting an activity or creating a shared challenge. Blocking and reporting belong in a consistently placed menu.

Challenge creation asks for title, invited participants, metric, target, start/end dates, timezone, and visibility. Support cooperative goals before emphasizing rankings. The detail screen shows progress, participating friends, remaining time, and accessible status labels. Explain how joining late, leaving, edited goals, and missing tracking data affect results. Invitations require acceptance; a sender must not publish another person's participation automatically.

5.8 Music experience

Music is optional and starts only after a deliberate user action. A compact player shows track or ambience title, play/pause, volume, mute, and source attribution. Expanded controls include mood, instrumental preference, adaptive mode, and **Keep this mood**. Offer a clear **Off** setting that persists across sessions.

Recommend music from the book's broad mood and the reader's choices. Adaptive changes should occur at sensible scene/chapter boundaries, avoid abrupt volume jumps, and respect user overrides. Do not inspect unread chapters to generate a mood label that reveals future events. Explain unavailable playback and external-provider sign-in without trapping the reader in an integration flow. Distinguish suggesting a playlist from having permission to play its tracks. Stop or pause according to explicit session preferences; do not surprise the user with audio when reopening a book.

Interface blueprint

Design language and themes

Quiet paper. Clear tools.

Semantic color, editorial type, restrained surfaces.



Paper
#F8F6F1

Ink
#202A25

Forest
#2D5B4B

Muted
#56635B

Focus
#2458A6

Light

A quiet chapter

The book remains central. Tools appear when the reader needs them.

Aa 20 px 1.7 line height

Sepia

A quiet chapter

The book remains central. Tools appear when the reader needs them.

Aa 20 px 1.7 line height

Dark

A quiet chapter

The book remains central. Tools appear when the reader needs them.

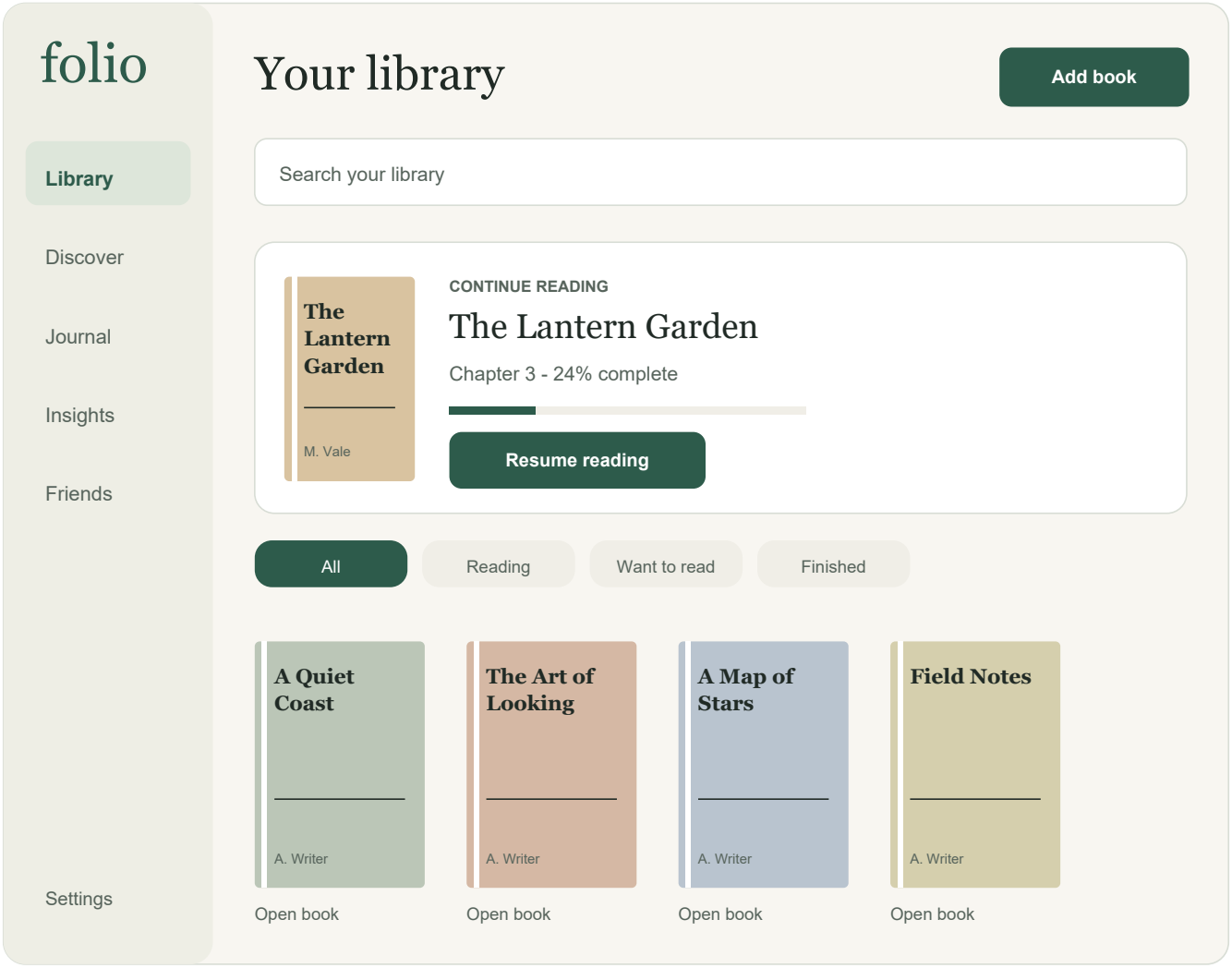
Aa 20 px 1.7 line height

Proposed colors and reading surfaces. Font choices in the product specification are Literata/Inter with language-appropriate alternatives; the manual uses locally available editorial fonts for these blueprints.

See Chapters 4-5 for dimensions, states, accessibility and release boundaries. These diagrams are design references, not working application screens.

Interface blueprint

Library / reading comes first

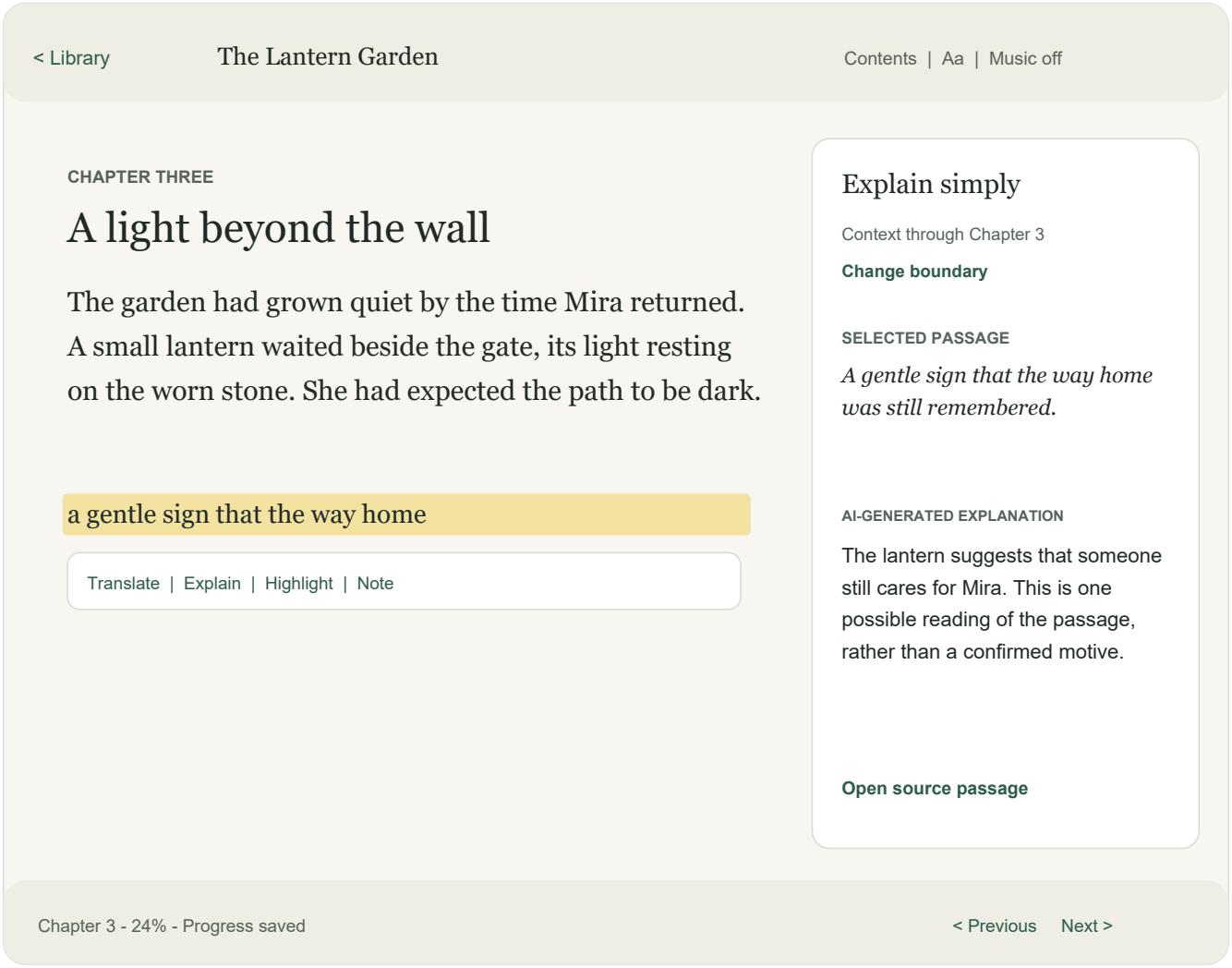


R1 primary screen. The Continue reading card leads; search and shelves support retrieval. Sample titles and progress are illustrative. Empty, processing and failed-import states are specified in Chapter 5.

See Chapters 4-5 for dimensions, states, accessibility and release boundaries. These diagrams are design references, not working application screens.

Interface blueprint

Reader / assistance beside the passage

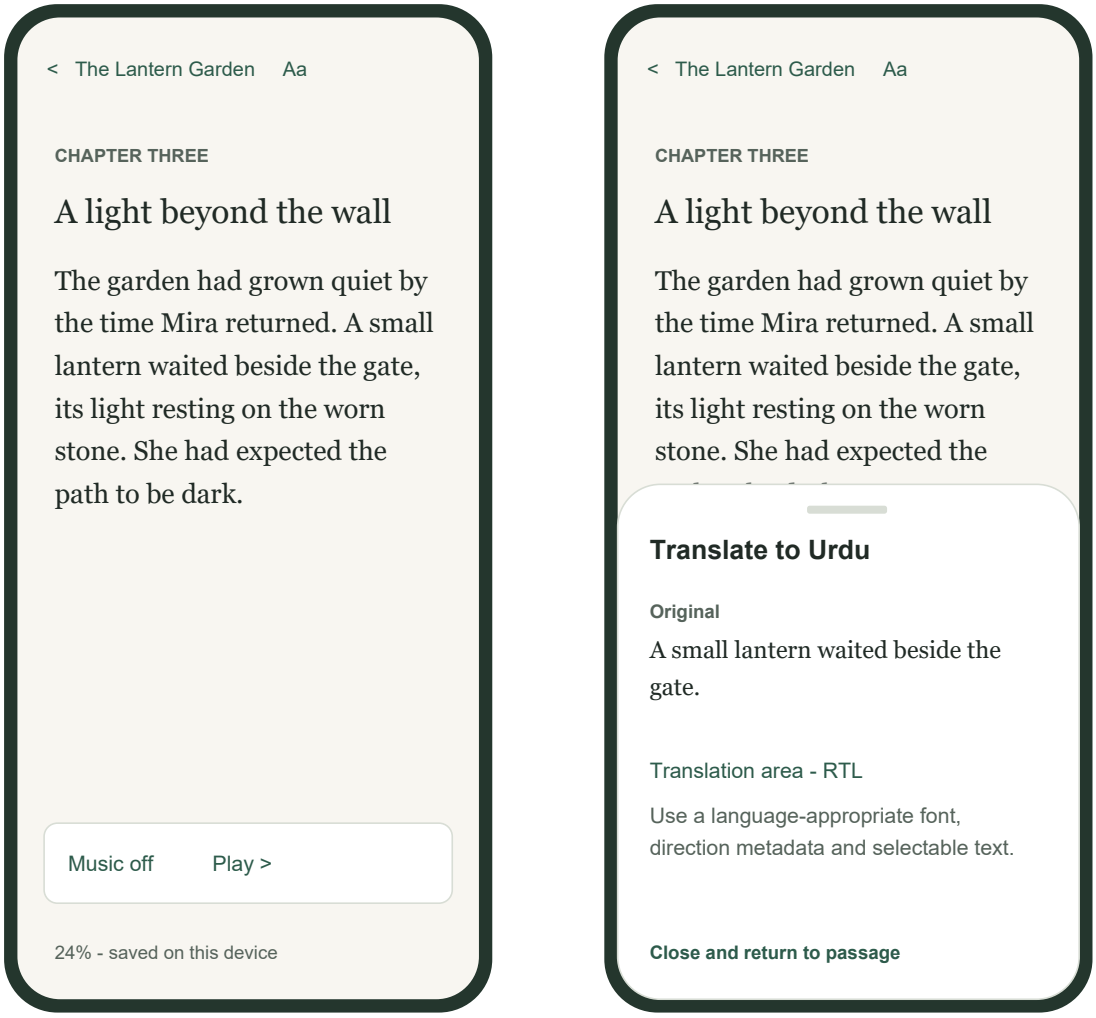


R1 reading shell with an explicit AI boundary. The selected passage stays in view. Music and scene illustration appear only in their enabled releases. Sample passage is original demonstration text.

See Chapters 4-5 for dimensions, states, accessibility and release boundaries. These diagrams are design references, not working application screens.

Interface blueprint

Mobile reader / focused assistance

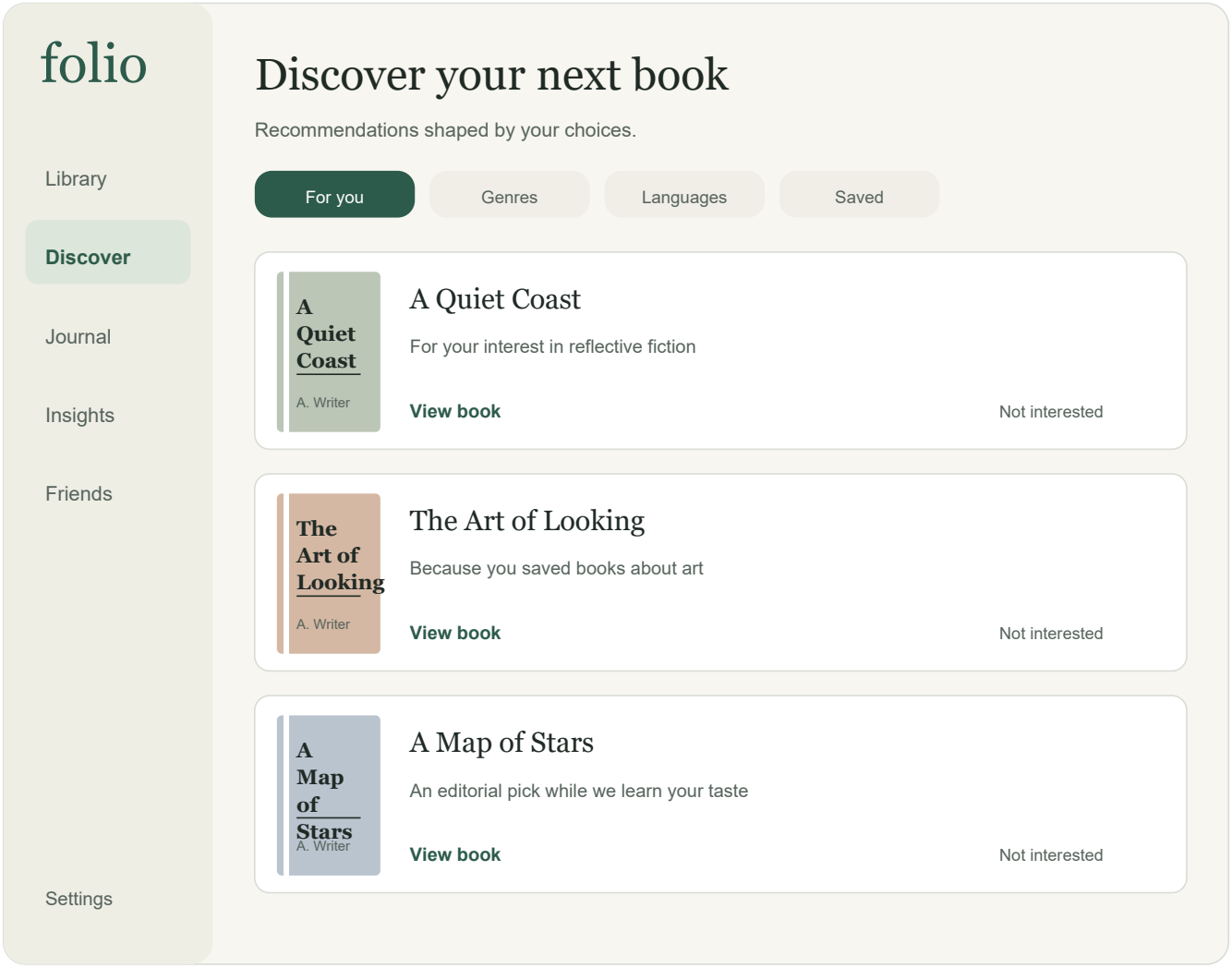


Proposed mobile interaction states. The translation sheet preserves reading position. RTL placeholder demonstrates layout only; a production Urdu translation requires a tested Arabic-script font and human review.

See Chapters 4-5 for dimensions, states, accessibility and release boundaries. These diagrams are design references, not working application screens.

Interface blueprint

Discover / reasons and control

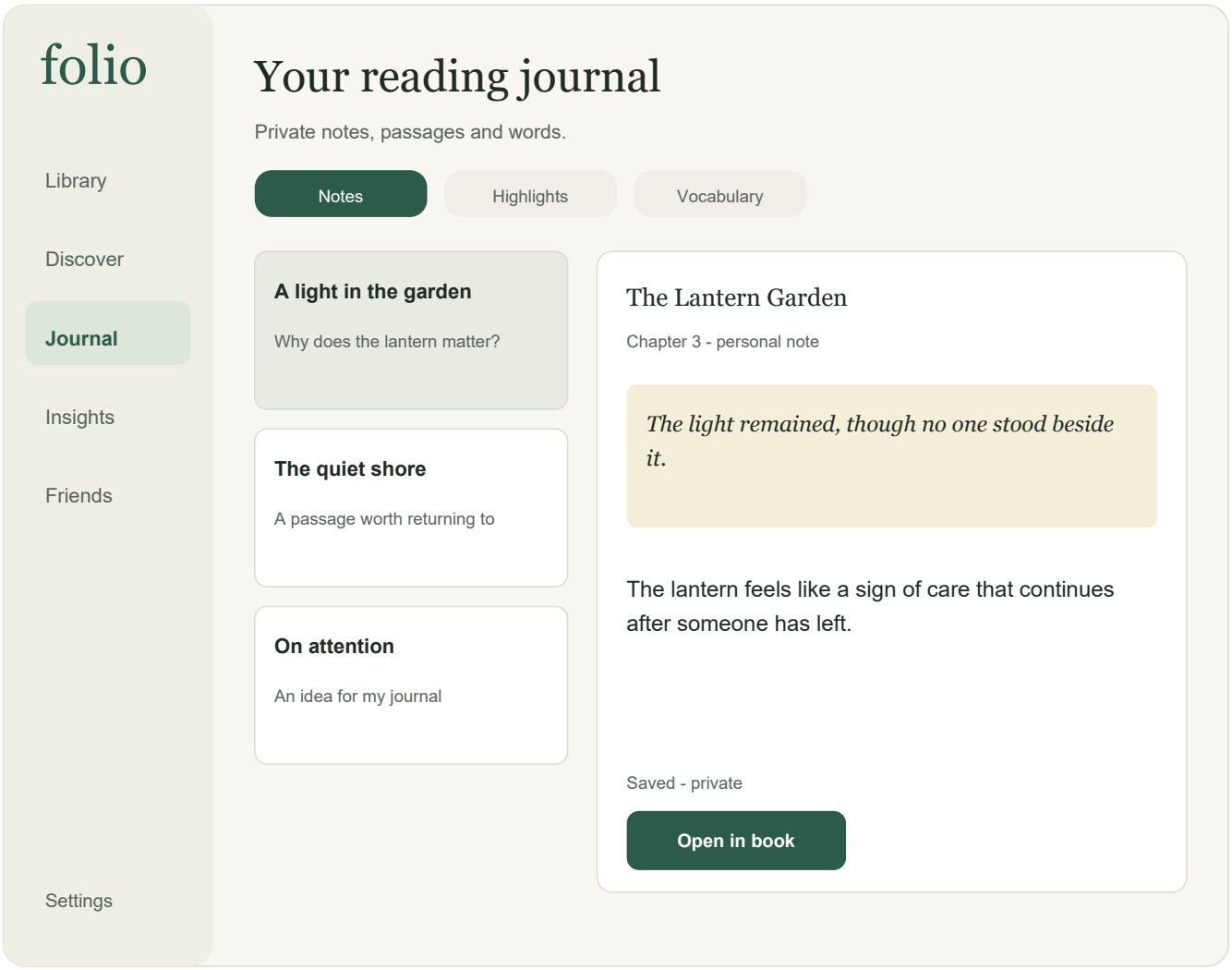


R2 expansion. Reasons come from actual preferences or history. Editorial suggestions are labeled; metadata access is distinct from permission to read a full book.

See Chapters 4-5 for dimensions, states, accessibility and release boundaries. These diagrams are design references, not working application screens.

Interface blueprint

Journal / private ideas in context

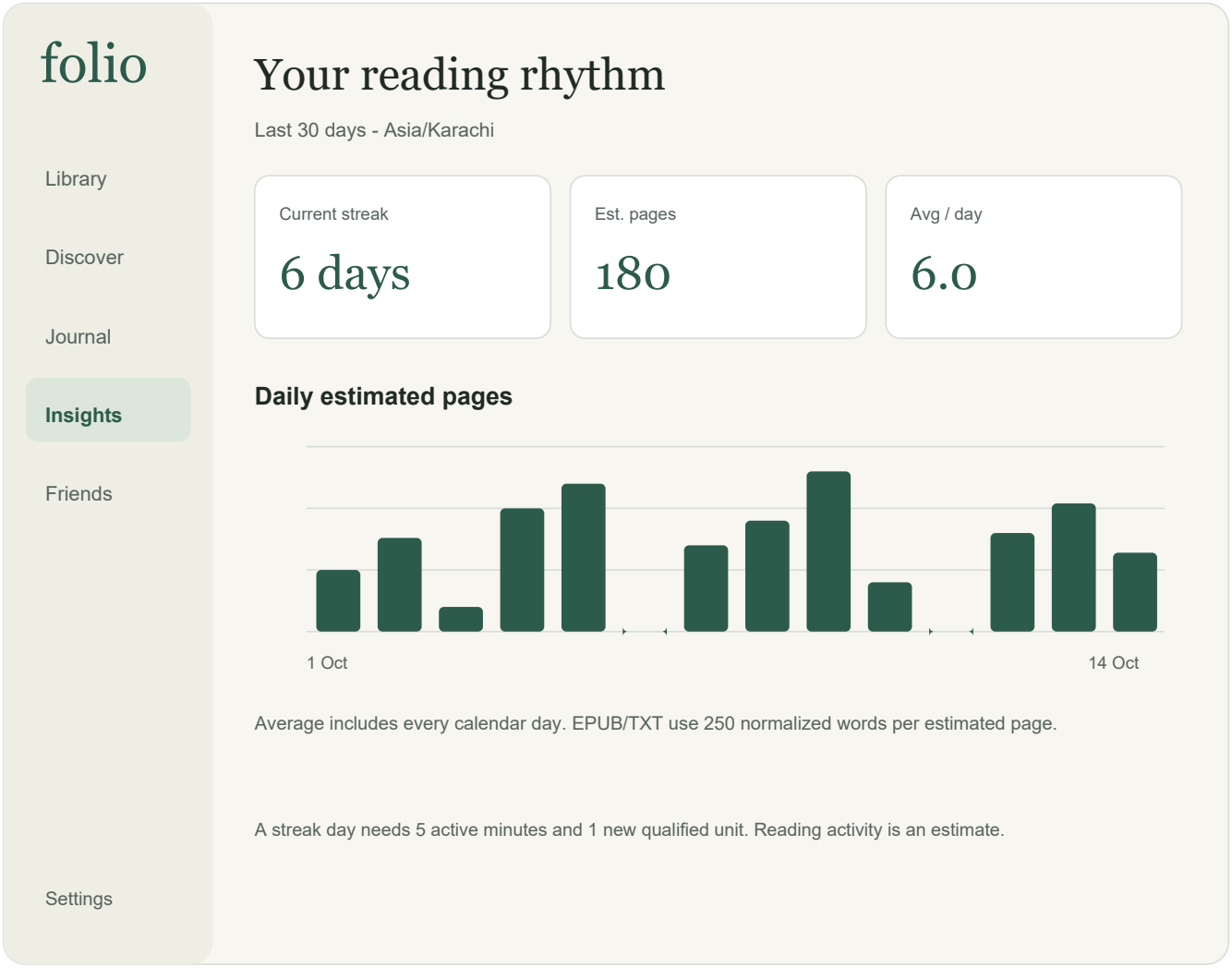


R1 notes and highlights; vocabulary arrives in R2. A list/detail arrangement keeps the selected annotation and source book together. Draft recovery and saved state are part of the component contract.

See Chapters 4-5 for dimensions, states, accessibility and release boundaries. These diagrams are design references, not working application screens.

Interface blueprint

Insights / understandable activity

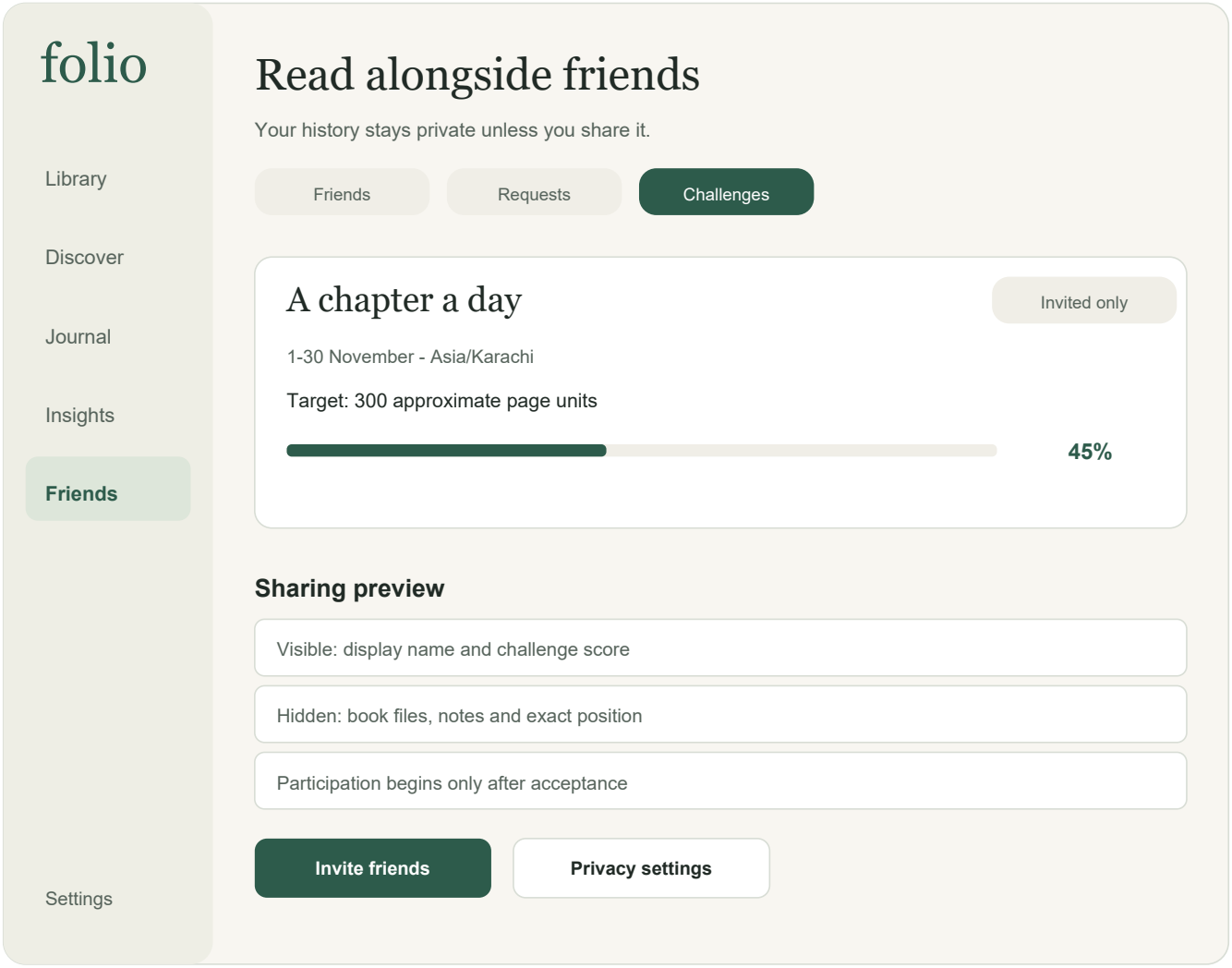


R2 expansion. These are sample metrics. State the date range, timezone, estimated-page convention and average denominator. Supply table alternatives for charts; the daily chart shows a portion of the selected period.

See Chapters 4-5 for dimensions, states, accessibility and release boundaries. These diagrams are design references, not working application screens.

Interface blueprint

Friends / sharing by choice



R3 expansion. Challenge participation is explicit. Display exactly what friends can see, freeze scoring rules, and enforce blocking independently of the interface.

See Chapters 4-5 for dimensions, states, accessibility and release boundaries. These diagrams are design references, not working application screens.

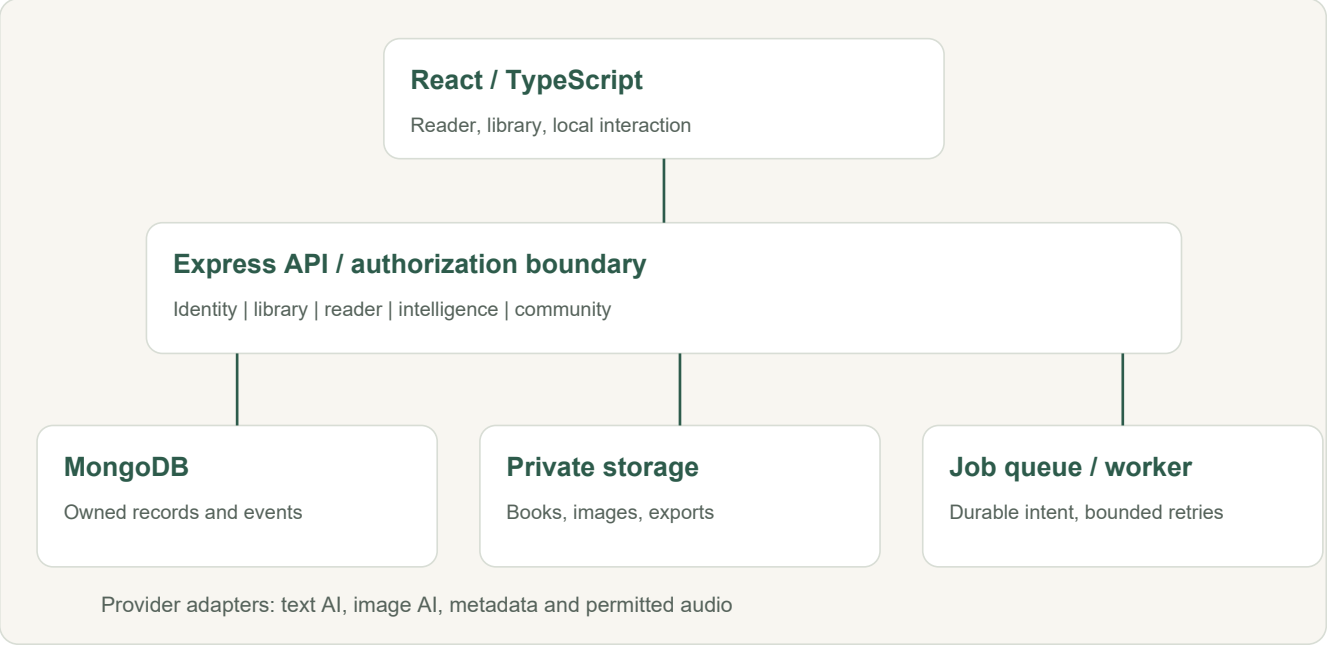
6. System architecture and service responsibilities

6.1 Start with a modular monolith

Use one TypeScript Express application with explicit domain modules and a separate background-worker process sharing domain code. This keeps deployment and debugging manageable while allowing slow work to leave the request path. Do not begin with microservices solely because the feature list is long.

The React frontend calls a versioned JSON API. The API handles authentication, authorization, validation, metadata, synchronization, budget checks and job submission. Workers process imports, derived text, image generation, exports, recommendation batches and optional recap updates. MongoDB stores structured records. Private object storage stores uploaded books, extracted assets, images, and exports. Redis can support queues and short-lived cache when asynchronous jobs become necessary; it should not be the only store of user progress or job intent.

6.2 Architecture map



Every privileged action crosses the API authorization boundary. Browser clients never receive storage administrator credentials, AI secret keys, or unrestricted provider tokens. Short-lived signed file access follows the library ownership check. The diagram shows logical responsibilities; production may use managed equivalents.

6.3 Frontend responsibilities

- Route shell and accessible navigation.
- Reader renderer adapters for EPUB, PDF and TXT.
- Transient interaction state such as selection, panel visibility, music controls and theme.
- Server data loading, mutation status and cache invalidation.
- Explicit offline package and synchronization queue when supported.
- Rendering user settings and server-calculated statistics without inventing values.

Keep reader location and annotations independent of rendered DOM component IDs. Treat a renderer adapter as an integration boundary: opening, locating, navigating, selecting, highlighting and reporting visible range should use an application-owned interface. This makes later renderer replacement feasible.

6.4 API module responsibilities

Module	Owns	Collaborates with
Identity	Users, sessions, privacy and preferences	Every protected resource
Library	Upload lifecycle, entries, document versions, rights provenance	Storage, import workers
Reader	Locations, reading events, progress, annotations	Library, statistics
Intelligence	Context policy, requests, outputs, budgets, evidence references	Allowed book chunks, AI adapters
Audio	Track rights, preferences, playback suggestions	Catalog and client player
Discovery	Taste inputs, catalog metadata, ranked results, feedback	Permitted reading history
Community	Friendships, blocks, circles, challenges, reports	Privacy policy and notifications
Platform	Jobs, outbox, audit events, telemetry, export/deletion workflows	All modules

Domain modules expose services, not arbitrary access to one another's collections. Authorization remains in both the request boundary and appropriate domain services so a background job cannot accidentally bypass ownership rules.

6.5 Recommended implementation choices

Use React with TypeScript and a production frontend build tool such as Vite. Use an accessible routing solution and a query cache for server state; local component state is sufficient for most transient UI state. Express with TypeScript owns the API. Use an object schema validator at every external boundary and publish an OpenAPI contract from reviewed schemas. Use MongoDB with either a typed native-driver layer or one ODM consistently; the proposed folder layout uses Mongoose as an optional implementation choice, not a requirement.

Evaluate epub.js for EPUB rendering and PDF.js for PDF. Verify maintenance, licensing and device behavior at implementation time. Neither should be treated as a sanitizer or a way to remove DRM. An import parser and a browser renderer have different responsibilities. For asynchronous work, evaluate a durable queue backed by Redis with persisted job intents/outbox records so business mutations are recoverable. Select maintained package versions at implementation time, commit a lockfile and pin the Node runtime used in CI.

Avoid introducing a vector database in R1. Retrieve bounded text chunks from the current document and enforce the spoiler cutoff before ranking. Add semantic retrieval only if useful explanations require it, with document ownership and progress filters that cannot be omitted.

6.6 Request and asynchronous job patterns

Quick lookups and metadata updates use ordinary requests. Translation and explanations can stream through an authenticated fetch response or use a normal complete response for the first release. With fetch streaming, support abort and a terminal event containing request status, usage and evidence references. Avoid placing bearer tokens or selected text in query strings.

Image generation and imports return `202 Accepted` with a durable job ID and status URL. Jobs follow queued, running, succeeded, failed, cancelled and expired states. Workers retry only transient failures with bounded backoff. A terminal provider failure should not cycle indefinitely or charge again on every browser refresh.

6.7 Deployment topology

Use the same public origin for the frontend and `/api` where practical, reducing session and cross-origin complexity. Serve versioned frontend assets through a CDN; keep user-specific responses private. Use separate development, staging and production databases, queues, storage buckets and provider credentials. Production has API processes, worker processes, a managed database and private storage. Add queue/cache infrastructure when the corresponding jobs ship.

7. Repository structure and engineering conventions

7.1 Proposed monorepo

```
folio/
  apps/
    web/
      src/
        app/                routing, providers, shell
        features/
          auth/ library/ reader/ journal/ intelligence/
          audio/ discover/ insights/ friends/ challenges/
        components/         shared accessible components
        design/             tokens, themes, global styles
        lib/                API client, storage, formatting
        locales/            navigation copy and message catalogs
      public/               permitted sample assets
    api/
      src/
        app.ts              Express composition
        server.ts           startup and graceful shutdown
        modules/
          identity/ library/ reader/ intelligence/
          audio/ discovery/ community/ platform/
        middleware/         session, validation, rate limits, errors
        config/             validated server configuration
    worker/
      src/                  queue consumers and job runners
  packages/
    contracts/              request, response and event schemas
    domain/                 shared server domain logic
    reader-core/            location and renderer interfaces
    ui/                     tokens and reused UI primitives
    tooling/                lint, TypeScript and test configuration
  docs/                     architecture decisions and runbooks
  tests/                    cross-application scenarios
  infra/                    environment and deployment definitions
  scripts/                  controlled migrations and maintenance
```

7.2 Module shape

Each API module separates routes/controllers, validation schemas, application services, repositories/models and authorization policy. Controllers translate HTTP; services enforce business rules; repositories implement persistence. Provider adapters are injected into services so mock adapters can be used in automated tests without presenting mock AI as a completed production feature.

Shared contracts contain serialization-safe types and validation schemas. They must not import database credentials or server-only implementations into the browser. Store dates in UTC ISO format over HTTP and retain a separate IANA timezone for calendar calculations.

7.3 Naming and quality rules

Use document versions and stable opaque IDs in persisted references. Prefer explicit names such as `currentLocator`, `maxReadLocator`, `aiContextBoundary` and `metricVersion` over ambiguous `page`. Treat user content as text unless a vetted rendering policy allows specific markup. Use structured error codes, request IDs and accessible human messages. Never expose stack traces in production responses.

Define feature completion as working UI, real persistence, authorization, error recovery, suitable verification and documentation. Update architecture decision records when assumptions change. Keep migration scripts repeatable and backups verified before data-shape changes.

8. Data model and ownership

Records use opaque IDs and UTC timestamps. Responses omit secrets, private object paths, and internal moderation fields. Paginate growing lists. These proposed collections arrive with their features; they need not all exist initially. Uniqueness and ownership invariants apply when their feature ships.

Keep original books, covers, extracted content manifests, scene images, and audio in private object storage. MongoDB stores their metadata and access relationships. Issue short-lived asset URLs only after checking the requester; removing a library item or membership must not leave permanent public file links. Keep temporary processing assets separate from durable user-owned outputs and delete abandoned uploads through scheduled retention jobs.

Collection	Proposed principal fields and indexes
`users`	`emailNormalized`, password hash or external identity, handle, display name, timezone, locale, privacy preferences. Unique normalized email and handle.
`authSessions`	`userId`, hashed refresh token/session identifier, expiry, revocation, device label. Index user and expiry; expiry cleanup complements explicit server expiry checks.
`books`	Work-level title, authors, genres, language, description, catalog source. Search indexes cover supported discovery fields.
`bookEditions`	`bookId`, format, content hash, `manifestVersion`, canonical word count/native page count, processing status, access policy. Index book and content hash within the appropriate ownership scope.
`contentChunks`	`editionId`, `manifestVersion`, ordinal, canonical start/end boundaries, permitted text or private text asset reference. Unique edition plus manifest version plus ordinal; index canonical boundaries for bounded retrieval.
`assets`	Owner, edition/job references, object key, MIME type, byte size, checksum, status. Index owner and lifecycle status.
`userBooks`	`userId`, `editionId`, shelf, rating, started/completed timestamps, current `readingPassId`, position version. Unique user plus edition.
`readingPasses`	`userBookId`, pass number, current locator, furthest qualified locator, spoiler boundary, completion state, revision. Unique user book plus pass number.
`annotations`	Owner, edition, anchored range, quote fingerprint, note/highlight, visibility, revision, deletion marker. Index owner, edition, and update cursor.
`readingEvents`	User, `readingPassId`, client event ID, device, observed time, accepted local date, validated ranges, active duration. Unique user plus client event ID.
`readingUnitCredits`	User, `readingPassId`, canonical unit ID, credit date, accepted event reference. Unique reading pass plus unit ID; index user plus credit date.
`dailyActivity`	User, local date, timezone snapshot, credited pages, active seconds, qualifying flag, aggregate revision. Unique user plus local date.
`aiRequests`	Owner, edition, bounded input references, task type, boundary hash, model/config revision, job status, result asset/reference, expiry. Index owner and creation cursor.
`aiOutputs`, `sceneGallery`	Optional separate outputs: owner, request, `editionId`, `manifestVersion`, authorized boundary, result/asset reference; gallery adds saved order and caption. Unique output per request/result ordinal; index owner plus edition plus creation time.

Collection	Proposed principal fields and indexes
`vocabulary`, `reviewStates`	Owner, normalized term, source/target language, anchored examples; review state adds due date, interval, difficulty and revision. Unique owner plus term plus language pair; unique state per vocabulary item; index owner plus due date.
`usageReservations`	User, operation key, provider attempt ID, reserved units, settled units, reservation/settlement state. Unique user plus operation key plus attempt ordinal; partial unique index on present provider attempt IDs. Keep uncertain attempts reserved until reconciled.
`jobs`	Owner, operation type, state, attempts, lease, idempotency key, result reference, sanitized error. Unique scoped operation key; index state plus next attempt time.
`outbox`	Durable server-side job/event intent, aggregate reference, operation key, payload reference, delivery state, attempt count, next attempt time. Unique operation key; index delivery state plus next attempt. Write atomically with the initiating mutation.
`notifications`	Recipient, type, minimal payload reference, deduplication key, created/read times. Unique recipient plus deduplication key; index recipient plus created time.
`syncTombstones`, `deviceCursors`	Owner, deleted object ID/revision, expiry; device ID, acknowledged cursor, last sync time. Index owner plus deletion cursor; unique owner plus device. Retain tombstones through the supported offline window.
`auditEvents`	Actor, action, target, request ID, UTC time, redacted metadata. Index target/time and actor/time; append-only with restricted administrative access and explicit retention.
`friendships`	Canonically sorted user pair, requester, pending/accepted status. Unique pair; index each member and status.
`blocks`	Blocker and blocked user. Unique directed pair; index blocked user for permission checks.
`circles`, `memberships`	Circle privacy/owner and member role/status. Unique circle plus user membership.
`challenges`, `challengeParticipants`	Rules, dates, timezone, eligible content, scoring version; participant snapshots and totals. Unique challenge plus user.
`challengeContributions`	Challenge, participant, credited unit/event reference, accepted score, scoring policy revision. Unique challenge plus participant plus credit reference.
`recommendationRuns`, `feedback`	Owner, input revision, ranked work IDs, explanations, exclusions, feedback type. Index owner and newest generation time.
`musicTracks`, `musicPreferences`	Track provenance, mood/tags, playback eligibility; user volume, exclusions, favorites. Index enabled tracks by mood and provider.

Use references for relationships and bounded embedded objects for settings or locator values. Declare upload ownership and entitlement independently of catalog identity: two users possessing a record for the same title does not authorize access to each other's files. Do not globally expose a private upload through deduplication.

Use transactions for small, tightly coupled changes such as friend acceptance and challenge enrollment. For event processing, the unique event and unit-credit indexes are the primary duplicate barriers. Aggregates are repairable projections of accepted records; a retry or nightly reconciliation must reproduce the same totals. Never rely on a mutable counter as the only evidence that a page was read.

9. HTTP API contracts

All routes below are proposed and use the `/api/v1` prefix. Require authenticated ownership checks by default; public catalog/profile access is deliberate and filtered.

Endpoint group	Operations and permissions
`POST /api/v1/auth/register`, `POST /api/v1/auth/login`, `POST /api/v1/auth/refresh`, `POST /api/v1/auth/logout`	Create, renew, and revoke sessions; rate-limit credential operations and prevent account enumeration.
`GET/PATCH /api/v1/me`, `GET/PATCH /api/v1/me/preferences`, `POST /api/v1/me/export`, `POST /api/v1/me/deletion`	Read/update own profile and privacy, request export or deletion; sensitive operations require recent authentication.
`GET /api/v1/books`, `GET /api/v1/books/:id`	Discover permitted catalog metadata; exclude inaccessible private uploads.
`POST /api/v1/uploads`, `POST /api/v1/uploads/:id/complete`	Authorize bounded upload, then verify size/type/checksum and begin processing; owner only.
`GET/POST /api/v1/library`, `PATCH/DELETE /api/v1/library/:userBookId`	Add/remove an accessible edition; update own shelf/rating; removing access triggers an explicit retention policy for notes.
`GET /api/v1/editions/:id/manifest`, `GET /api/v1/editions/:id/content`	Read authorized edition manifest or bounded content/assets; check entitlement on every access.
`POST /api/v1/reading-passes`, `PATCH /api/v1/reading-passes/:readingPassId/position`, `PATCH /api/v1/reading-passes/:readingPassId/spoiler-boundary`	Create explicit pass, save versioned resume position, change authorized boundary; owner only.
`POST /api/v1/reading-events/batch`, `GET /api/v1/sync?cursor=...`	Submit idempotent activity batches; reconcile owned annotations and state changes through a stable cursor.
`GET/POST /api/v1/annotations`, `PATCH/DELETE /api/v1/annotations/:id`	Create/edit/delete anchored notes and highlights; owner writes, readers require explicit sharing permission.
`POST /api/v1/ai/translations`, `POST /api/v1/ai/explanations`, `POST /api/v1/ai/scene-images`	Validate ownership, bounded selection, budget and spoiler policy; return immediate result or `202` job reference.
`GET /api/v1/jobs/:id`, `POST /api/v1/jobs/:id/cancel`	Inspect/cancel own jobs when cancellation is still possible; never expose another user's inputs.
`GET /api/v1/music/recommendations`, `GET/PATCH /api/v1/music/preferences`, `POST /api/v1/music/tracks/:id/playback`	Return eligible tracks and authorized playback data; user initiates browser audio.
`GET /api/v1/recommendations`, `POST /api/v1/recommendations/feedback`	Retrieve personalized candidates and submit own dismiss/like feedback.
`GET /api/v1/activity/summary`, `GET /api/v1/activity/days`	Read own streaks, units, active time, and explicitly defined averages.
`GET /api/v1/users/search`, `GET /api/v1/users/:id/profile`	Respect discoverability, blocks, and field-level profile visibility.

Endpoint group	Operations and permissions
`POST /api/v1/friends/requests`, `POST /api/v1/friends/requests/:id/respond`, `DELETE /api/v1/friends/:id`	Request friendship, accept/decline as recipient, remove as either member.
`GET/POST /api/v1/circles`, `GET/POST /api/v1/circles/:id/members`, `DELETE /api/v1/circles/:id/members/:userId`, `GET/POST /api/v1/challenges`, `POST /api/v1/challenges/:id/join`	Enforce membership, role, invitation, and challenge eligibility policies server-side.
`GET/POST /api/v1/blocks`, `DELETE /api/v1/blocks/:id`, `POST /api/v1/reports`, `GET /api/v1/notifications`, `PATCH /api/v1/notifications/:id`	Manage own blocks/reports and read own notifications; moderator operations use separate audited privileges.

Use `401` for missing/expired authentication, `403` for permitted-to-identify but forbidden actions, `404` where revealing existence would leak private content, `409` for revision conflicts, `422` for invalid domain input, and `429` for quotas. Async responses expose job states such as queued, running, succeeded, failed, and canceled, with a recoverable error message.

Successful responses use `{ data, meta: { requestId, nextCursor? } }`; failures use `{ error: { code, message, fieldErrors? }, meta: { requestId } }`. Stable error codes drive interface behavior; provider stack traces remain internal. Collection endpoints accept bounded `limit` and opaque `cursor` values, with a stable secondary sort by ID.

Creation requests that can incur cost or duplicate a logical action carry an `Idempotency-Key`. Scope it to user, operation, and request hash; reusing it with different input returns a conflict. Revision-sensitive updates carry `expectedRevision`; a conflict response includes the current revision and permitted recovery choices. The API derives the user ID from authentication, never from a trusted-looking request field. Validate both the route object and referenced IDs: owning a reading pass does not authorize a caller to attach somebody else's edition or note.

9.1 Example request contracts

These short examples use placeholder IDs. Each modifying request includes authentication and an idempotency key. The server resolves source text and allowed spoiler scope from owned records, rather than trusting client-supplied permission claims.

`POST /api/v1/ai/explanations`:

```
{
  "editionId": "ed_123",
  "manifestVersion": 1,
  "readingPassId": "rp_123",
  "selection": {
    "start": { "spineIndex": 2, "textOffset": 450 },
    "end": { "spineIndex": 2, "textOffset": 680 }
  },
  "language": "en",
  "style": "plain"
}
```

`POST /api/v1/reading-events/batch`:

```
{
  "deviceId": "device_123",
  "events": [{
    "clientEventId": "evt_123",
    "sequence": 7,
    "editionId": "ed_123",
    "manifestVersion": 1,
    "readingPassId": "rp_123",
    "observedAt": "2026-10-23T12:00:10Z",
```

```
      "timezone": "Asia/Karachi",
      "activeIntervals": [{
        "startAt": "2026-10-23T12:00:00Z",
        "endAt": "2026-10-23T12:00:10Z"
      }],
      "exposures": [{ "unitOrdinal": 2, "visibleWordRanges": [[500, 725]], "activeSeconds": 10 }]
    }]
  }
```

Word ranges are half-open normalized source-word offsets. This ten-second report can credit a unit, but cannot alone satisfy the five-minute streak rule. The server validates intervals/exposures and checks timezone against the applicable account snapshot.

`POST /api/v1/challenges`:

```
  {
    "title": "November reading circle",
    "visibility": "invite_only",
    "timezone": "Asia/Karachi",
    "startsOn": "2026-11-01",
    "endsOn": "2026-11-30",
    "metric": "creditedPageUnits",
    "target": 300,
    "rules": {
      "allowManual": false,
      "allowRereads": true,
      "countBeforeJoining": false,
      "lateSyncHours": 168
    }
  }
```

Challenge dates are inclusive local dates in the declared timezone. The server validates dates, limits, invitation rights, and the scoring policy before freezing the rules.

10. Reading progress, daily statistics and offline consistency

10.1 Canonical positions and spoiler boundaries

Every locator identifies an immutable `editionId` and `manifestVersion`. For EPUB/TXT, identify the spine/document entry, normalized text offset, and a short quote fingerprint; keep an EPUB CFI when available as an additional anchor. For PDF, identify the zero-based native page and normalized rectangle/text offsets. The server validates locations against the edition manifest. Font size, browser width, and reader theme never change canonical positions.

Changing the source file creates a new edition or manifest migration. Do not silently attach existing notes to changed text. During migration, use verified matching anchors; unresolved annotations become visibly unplaced and remain recoverable.

Maintain three distinct positions:

1. **Current position:** where the reader resumes. Moving backward changes this value.
2. **Furthest qualified position:** the greatest position reached through accepted reading activity. It is monotonic within a reading pass, but is not proof that all earlier content was read.
3. **Spoiler boundary:** the maximum continuous content range the user has authorized AI to use. Start conservatively and advance through consecutively qualified units or an explicit user confirmation. A jump to the final chapter must not automatically expose the intervening story to AI.

The UI offers a clear "Allow discussion through this point" action and a reset/reduce control. Explain that a boundary reduction affects future requests, while previously viewed answers remain in history until removed. The server must enforce the boundary on every request, including retrieval, summaries, music mood analysis, scene-image prompts, recommendation explanations, and cached results. Reject selections beyond the allowed boundary or request explicit confirmation through the UI.

Never send the full novel to a provider and rely on a prompt saying "avoid spoilers." Retrieve only permitted source chunks and omit future chapter titles or plot-bearing metadata. Cache keys include edition, task, language, allowed range, and configuration revision; private results are owner-scoped. Lowering a boundary invalidates incompatible cached responses. A generated image can still reveal information inferred by a model; position this as a best-effort safeguard and provide a report action.

10.2 Reading metrics that remain consistent

Define these rules before implementing dashboards or challenges:

- **PDF pages:** native document pages, independent of zoom. Exclude cover/front matter only when the edition manifest explicitly classifies them and the UI explains the denominator.
- **EPUB/TXT estimated pages:** fixed units of 250 normalized source words, with the final partial unit included. This is a product convention, not publisher pagination. Display an approximation label wherever these units appear.
- **Credit qualification:** record a unit once per reading pass after the client reports sufficient foreground exposure and active dwell. Initial policy: cumulative visibility of at least 80% of its text words, or 60% of a PDF page area, with at least eight active seconds. Treat this as activity estimation, not proof of comprehension. Accessible narration may provide an equivalent verified playback path.
- **Manual entry:** if offered, label manually logged pages and keep their source. Decide explicitly whether each challenge accepts manual activity; do not mix it into verified-by-app activity without disclosure.
- **Rereading:** revisiting an already credited unit does not add pages within the same pass. Starting an explicit rereading pass creates new credits, while preserving the earlier pass and history.

- **Active time:** count foreground reader activity or accessible narration playback, pause after inactivity, and merge overlapping intervals across devices. Reading music alone adds no reading time. Two simultaneous devices must not double the user's active seconds.

Daily streak: a day qualifies when it contains both at least five active reading minutes and at least one newly qualified unit for a reading pass on that local date. A current streak is consecutive qualifying local dates ending today or yesterday; yesterday remains current until today ends. The longest streak is the greatest run in retained history. An open book without qualified reading adds no streak credit.

Average pages per day: default dashboard value is credited page units in the last 30 local calendar dates, including today, divided by 30. For accounts younger than 30 days, divide by the number of local calendar dates since signup, including today. Include zero-reading days. A separate "per active reading day" metric divides by dates containing at least one credited unit and carries that exact label; it does not use the stricter streak threshold. Mixed EPUB/PDF totals are approximate combined page units and must be labeled accordingly; book-specific reports retain native/estimated distinctions.

Store UTC instants, the accepted local date, and timezone snapshot on activity. User timezone comes from a profile setting and can be changed with confirmation; do not silently follow every device's timezone. Changes apply prospectively and do not rebucket old history. Challenge dates use a fixed published timezone and a versioned scoring policy.

Offline events retain their observed timestamp, device sequence, timezone snapshot, and unique ID. Accept backdated streak credit only within a documented seven-day sync window; retain older activity separately without silently changing competitive scores. Flag implausible future timestamps or impossible durations for correction. Challenges publish their own sync cutoff. These are transparent product rules, not a claim that client activity is tamper-proof.

10.3 Offline and concurrent updates

The client outbox assigns immutable operation IDs before sending. The server returns per-operation acceptance, duplicate acknowledgment, or rejection so one invalid item does not lose an entire batch. Activity events are append-only; qualified credits use their unique indexes. Retries must not regenerate images, charge twice, or increment streaks twice.

Use optimistic revisions for positions, annotations, and preferences. If a second device has newer progress, preserve both device positions and offer a resume choice; do not choose the numerically furthest location because the user may intentionally be rereading. For note conflicts, keep recoverable versions and ask which text to retain. Deletion markers prevent an offline device from resurrecting removed notes. A reset reading pass is an explicit event, never an accidental result of a late sync.

Persist an AI job before invoking a billable provider. Reuse its scoped idempotency key and provider operation ID across retries where supported. Where a provider cannot guarantee deduplication, reconcile uncertain submissions before retrying. Failed or canceled jobs cannot attach orphaned results to a deleted book; retention jobs clean those assets.

11. Contextual AI, translation and scene generation

11.1 Context assembly is a server responsibility

Each request identifies the authenticated user, library entry, immutable document version, selected range, action, target language and requested explanation mode. The API resolves the selection against trusted extracted text. It verifies ownership, checks that the selection lies within the permitted boundary, estimates cost, and retrieves only eligible context.

The client must not be able to expand permissions by submitting a later chapter or a fabricated progress value. User-supplied text is allowed for a clearly labeled general explanation mode, but it must not pretend to be verified book content.

11.2 Spoiler policy and its limits

Maintain three separate concepts: current reading position, furthest validated reading position, and explicit AI context boundary. The user may keep the AI boundary behind their current position, expand it after a deliberate jump, or opt into whole-book discussion. The server caps evidence retrieval before sending anything to a provider. Recaps, character facts, images, music mood inference and recommendation explanations use the same policy.

Filtering retrieval alone cannot guarantee a foundation model will never use remembered knowledge of a famous book. Instruct it to rely only on supplied evidence, refuse requests requiring future text, inspect outputs for unsupported names/events where feasible, and offer a strict mode limited to evidence-backed paraphrase. Label the limitation honestly. Do not market an absolute spoiler guarantee that the system cannot verify.

11.3 Translation behavior

For a selected word, show contextual meaning, the translated form and an optional short usage explanation. For a sentence or paragraph, translate the complete selection and preserve formatting where useful. Let the user choose English, Urdu, Arabic, Persian, Spanish, French or any supported provider language. Support mixed-direction text in independent blocks.

Use a low-latency dictionary or local lexical cache for eligible common words; contextual translation can use a model or translation service. Distinguish a dictionary definition from a context-aware interpretation. Cache keys include document version, selection hash, action, target language, context boundary, provider configuration and prompt version. Private-content cache scope must remain per user or per explicitly authorized document scope. Do not share upload-derived translations between accounts by default.

When the provider fails, keep the original text visible and offer retry. A dictionary fallback must be labeled. Do not fabricate a successful translation. Pronunciation can be added through a permitted speech source; the UI should show when audio is unavailable.

11.4 Explanation modes and response contract

Modes are Explain simply, Explain deeply, Why this matters, Read between the lines, Give an example and Ask a question. Render a short answer first with expandable detail. Distinguish textual facts, plausible interpretations and uncertainty. Keep examples outside the book clearly identified as examples.

A normalized response includes the request ID, mode, answer blocks, evidence ranges, detected language, uncertainty note when needed, allowed context description, status and usage summary. Evidence links return to book locations without publishing book text. Do not expose hidden model reasoning; provide a concise explanation and supporting passages.

11.5 Prompt and input safety

Treat uploaded text, catalog metadata and notes as untrusted content. A sentence in a book that says to ignore instructions must remain book text. Separate system instructions, user request and book evidence. Give the explanation workflow no file-system, messaging, purchase or unrestricted browsing tools. Provider output is untrusted too: render sanitized text/markdown, validate structured JSON, and allow only safe links.

Limit selected-text length and context tokens. Redact unnecessary account identifiers before provider calls. Personal notes are excluded unless the user chooses to include a specific note. Record prompt/configuration versions and non-sensitive request metadata for diagnosis without logging the full private passage by default.

11.6 Scene visualization pipeline

The user chooses Visualize this scene, a style and optional context preference. Before submission, show a short scene description that can be edited. The API validates the eligible passage, reserves the generation budget and creates a job. A worker builds a visual brief using only allowed evidence, calls the selected image provider, stores the image privately and completes the job. The gallery records source locations, style, generation date and a label indicating that the image is an interpretation.

Character consistency is best effort. Store user-approved appearance notes tied to the progress boundary, and optionally use permitted prior images as references. Avoid a universal character profile containing facts from later chapters. Do not promise perfect identity continuity across unrelated generations.

Cancellation may stop local processing but not an already accepted provider request. Quota settlement must reflect provider semantics. Failed or deleted images must not reappear through a stale CDN link; use private access and expiration.

11.7 Story Memory, Character Compass and vocabulary

Story Memory summarizes eligible events with chapter references. Invalidate or recompute it when the allowed boundary changes. Character Compass stores evidence-backed facts and relationships with first-seen locations, then filters by the reader's boundary. Multiple interpretations can coexist if labeled.

Vocabulary entries store word, language, original context, translation, meaning and source range. Add an optional spaced-review schedule. Review sessions show a manageable set and allow skip or delete. Never make vocabulary practice a prerequisite to reading. A user can export notes and vocabulary without exporting another person's book text.

11.8 AI budgets, quality and evaluation

Use per-user daily quotas, maximum context size, concurrency limits, provider timeouts and an operator spend ceiling. Cache eligible results; reserve expensive-job budget atomically before submission; settle usage once per provider attempt. Keep an auditable distinction between requests, attempts, reservations and charges. Retry a request with the same idempotency key without creating an unintended second purchase of generation capacity.

Before release, evaluate a permitted set of passages across languages, figurative writing, non-fiction and ambiguous references. Score factual grounding, translation usefulness, spoiler leakage, handling of instructions embedded in books and graceful refusal. Humans should review representative multilingual and literary outputs. User feedback should link to the exact output and configuration, with permission before including private text in support review.

12. Music and adaptive reading audio

12.1 First music release

Start with a small permitted soundtrack catalog: instrumental, ambient, nature and quiet piano selections where rights allow the planned playback model. Music metadata includes title, creator, source, rights reference, allowed territories, expiry, attribution and playback method. Remove or disable an unavailable track gracefully.

The mini-player offers Play/Pause, volume, title, soundtrack picker and a clear Off state. Use explicit user interaction to begin playback because browsers restrict autoplay. Preserve reading progress while the player loads or fails. Make playback state independent of AI panels and route changes within the reading session.

12.2 Recommendation inputs

Use explicit preferences first: instrumental or vocals, energy, ambient sounds and volume preference. Then consider the user's chosen genre and the mood of an eligible passage. Offer a reason such as Quiet instrumental for reflective reading. Do not infer sensitive emotional or health states from a user's private book history.

Avoid sudden suggestions every paragraph. Prefer a soundtrack selected at session start with an optional suggestion at a chapter boundary. Let the user lock a track, skip a suggestion or choose silence.

12.3 Adaptive mode

Adaptive mode is opt-in and starts after user-initiated playback. Compute a coarse mood over a stable passage window, debounce changes, require a confidence threshold and enforce a proposed minimum five-minute dwell time. This duration is a UX starting point to test. Crossfade only when both the source rights and browser playback mechanism support it; otherwise change at a natural track boundary or suggest a manual switch.

Never analyze unread chapters to predict future mood. Avoid action-heavy changes that distract from reading. Provide reduced transitions and no automatic volume increases. A low-confidence mood or provider error leaves the current track unchanged.

12.4 External integrations and fallback

An external music integration requires a reviewed provider adapter, current playback permissions, account handling where required and regional availability checks. Do not assume a search API permits full playback, downloading, offline storage or Web Audio processing. Do not mix a provider's preview URL into an unrestricted soundtrack catalog.

If an integration is unavailable, offer a permitted local/catalog track or silence. Do not silently replace it with copyrighted audio from an unrelated service. Keep user audio preferences and music history optional and deletable. Offline reading does not imply offline soundtrack rights.

13. Personalized book discovery

13.1 Catalog and private-library separation

Discover operates primarily on book metadata, licensed descriptions, permitted covers and availability links. A private upload may be matched to a catalog record using metadata and explicit user confirmation; its file, annotations and full text remain private. A recommendation links to a legitimate source or an owned library entry.

13.2 Cold start and explicit control

With no history, ask optionally for three liked books, selected genres, preferred languages and desired difficulty. Otherwise show clearly labeled editorial collections. Do not invent a personalized explanation before a real signal exists.

User controls include Not interested, Already read, Save for later, More like this, Exclude author/genre, and Reset recommendations. Provide a history-use toggle and allow recommendations based only on explicit tastes. Account deletion removes the personal taste profile and feedback according to the deletion workflow.

13.3 First ranking model

Begin with a deterministic content-based score. A proposed starting mix is 30% theme match, 20% genre match, 15% author/style affinity, 15% language and difficulty fit, 10% explicit similar-book feedback, and 10% diversity/novelty. These are tuning defaults, not a trained or validated model. Filter hard constraints, blocked authors, duplicates and unavailable content before ranking.

Apply a diversity pass so one popular author does not occupy the whole page. Explain a result using actual contributing signals: Because you enjoyed reflective historical fiction and saved similar titles. Avoid claims based on inferred behavior that was never recorded. Do not reveal plot details beyond the user's permitted progress in comparison copy.

13.4 Evolving the model

After enough consented interactions exist, compare the baseline with improved content embeddings or collaborative signals. Use offline relevance review, explicit feedback, catalog coverage and diversity measures. Track saves followed by successful reading rather than clicks alone. Keep a simple fallback for outages and users who opt out.

Record recommendation request ID, model/rule version, signal categories, ranked IDs and reason codes. Do not store raw passages in the recommendation log. Distinguish a promotional placement from a personalized recommendation if monetization is introduced.

14. Friends, privacy and challenges

Default uploaded books, annotations, history, and exact reading positions to private. Friend acceptance grants only the fields the user elects to share. Challenge membership reveals the agreed score, not book files, private notes, or exact passages. Blocking removes discovery, requests, invitations, direct activity visibility, and future shared notifications in both directions; define how existing circle moderation remains available.

Challenge creation freezes the scoring rules, calendar timezone, date range, allowed activity sources, reread eligibility, and late-sync cutoff. Joining is explicit. Score only eligible credits within that window and deduplicate contributions; joining the same challenge twice cannot double the score. State whether pre-join activity counts before enrollment. A recommended default counts activity after joining. Publish tie handling and withdrawal behavior; show limited profile information on leaderboards and let members leave a challenge without exposing their history.

Friend requests have one pending or accepted relationship per unordered pair. Only the recipient can accept; either user can decline/remove, and repeated request attempts are rate-limited. A pending request does not grant access. Recheck block and visibility rules when serving a feed item, rather than trusting the audience captured when it was first created.

Enforce privacy in queries and serializers, not only by hiding interface controls. Notification previews, exports, leaderboards, caches, analytics, and recommendation explanations follow the same policies. Account deletion revokes access immediately and queues deletion of personal files and provider artifacts under the published retention policy; preserve only necessary de-identified aggregates or records with a stated retention reason.

15. Security, privacy and content handling

15.1 Authentication and access control

Use a reviewed authentication implementation or identity provider. For a same-origin browser app, prefer secure, HttpOnly, appropriately SameSite session cookies with server-side revocation, rotation and expiry. Protect state-changing requests against cross-site request forgery using origin checks and a token mechanism suitable for the deployment. Configure credentialed CORS to a specific allowlist if separate origins are unavoidable.

If passwords are handled by the application, use a vetted password hashing implementation, one-time expiring reset tokens, email verification where needed, and rate limits that resist enumeration and abuse. Add MFA for operators. Never put session secrets in browser local storage or URL parameters.

Every resource lookup checks the authenticated owner's access, not only whether an ID is hard to guess. Apply explicit permissions to library entries, annotations, AI requests, images, exports and job status. Block relationships override friendship, challenge and activity-sharing permissions.

15.2 Upload and rendering security

Validate actual format as well as extension and declared MIME. A proposed initial upload limit is 50 MB per EPUB, configurable after load testing. Limit expanded archive size, entry count, compression ratio, image dimensions, parsing time and XML processing; reject path traversal, nested archive abuse and unsafe external references. Quarantine uploads until validation and any required scanning complete.

Disable EPUB scripting and remote resource loading by default, sanitize content, and render in a restricted context with a deliberate content security policy. Treat EPUB CSS and fonts as untrusted inputs too. PDF rendering should use an updated renderer with scripting/actions disabled where applicable. Files requiring DRM removal are unsupported; never ask users to bypass it.

Keep raw files in private object storage. Object keys must not act as authorization. Use short-lived signed access, least-privilege service roles, encryption in transit and at rest, secret rotation and separate environment credentials. Prevent the import service from fetching arbitrary URLs supplied by a book; if acquisition URLs are later supported, constrain them through a reviewed server-side fetch policy.

15.3 Privacy and data lifecycle

Default library, notes, history, vocabulary and AI gallery to private. Expose granular settings for public profile fields, friend activity, statistics, challenges and history-based discovery. The default provider payload contains only the minimum passage context needed; including notes is a separate explicit choice.

Publish an accurate explanation of provider processing and actual retention settings before enabling AI. Do not promise zero retention unless the selected provider agreement and configuration support it. Keep application logs free of complete book text, private notes, session tokens and signed URLs.

Proposed retention defaults for review: upload-staging objects expire within 24 hours after terminal processing; temporary exports expire within seven days; detailed operational request metadata expires after 30 days unless needed for an active incident; aggregate deidentified service metrics can be retained longer. Users can delete generated content sooner. These are proposed policy settings to validate, not confirmed legal requirements.

15.4 Deletion and export

Deletion is a tracked workflow: disable access, revoke sessions where relevant, cancel pending work, delete or detach user records, purge private objects and derived AI outputs, invalidate caches, and confirm completion status. Handle failed steps with retries and operator visibility. Retain only records required by an actual policy or obligation, documented to the user.

Backups require a published expiration period and a process that reapplies deletions after restore. Do not claim that a deletion instantly removes immutable backups. Exports verify the requester and include portable annotations, reading history, settings and vocabulary in documented formats. Distribute book files only when the user's rights and export scope permit it.

15.5 Social safety and moderation

Provide block, mute, report, private challenge membership and rate limits before launching community features. Private notes are never automatically posted. A moderator can review reported public/shared content with an audit trail. Public discussion systems need rules for harassment, unwanted invitations and copyrighted excerpts as well as spoiler labels.

16. Performance, accessibility and resilience

16.1 Proposed performance budgets

Interaction	Proposed target	How to assess
First useful app view	LCP at or below 2.5 s at the 75th percentile on an agreed device/network profile	Browser field measurements and synthetic checks
Interaction responsiveness	INP at or below 200 ms at the 75th percentile on the same profile	Field measurement with user-content redaction
Reader navigation after content loads	Visible response within 150 ms for ordinary EPUB chapter/page movement where feasible	Measured renderer scenarios
Cached lookup	API response within 300 ms at p95 in the deployment region	Service histograms, excluding client network where stated
AI explanation	Loading/stream feedback immediately; first content target within 3 s	Provider-specific measurements; no guaranteed completion time
Import or image job	Acknowledge within 1 s under normal load; show live durable status	Separate queue delay from processing duration
Position persistence	Local save within 1 s; online sync shortly after a stable change	Controlled disconnection and restart scenarios

These are budgets to refine against real devices, accessibility tooling, content and provider limits. A large illustrated EPUB or scanned PDF may behave differently. Report targets and observed measurements separately.

16.2 Optimization boundaries

Load reader code only on reader routes. Load expensive PDF and image tooling on demand. Parse/import in background workers. Paginate libraries, journals and social lists. Index real query shapes; use projections and keyset pagination for long histories. Keep canonical book chunks immutable by version.

Cache static metadata and permitted public assets separately from private user responses. Compress text responses and stream larger assets. Large private files should not pass through the API process unnecessarily after authorization. Monitor queue latency, memory per parser, database query times and provider response times before adding infrastructure.

16.3 Accessibility acceptance

Target WCAG 2.2 AA for supported flows and verify with automated checks plus keyboard and assistive-technology testing. Use semantic landmarks, visible focus, labels, predictable tab order, meaningful headings and descriptive action names. Respect reduced motion. Offer reflow, large text and high-contrast settings. Avoid relying on color alone for highlight categories or challenge progress.

The text-selection toolbar must have an alternate keyboard-accessible path. An explanation panel announces loading and completion without repeatedly reading streamed fragments. Dialogs trap focus only while modal, close predictably, and return focus to the triggering control. Test touch targets, 200% text resizing, narrow viewports and mixed RTL/LTR reading.

16.4 Offline behavior and honest state

Mark the exact books downloaded for offline reading. Show what is available offline, the last synchronized time and whether new notes are only on this device. AI and external media can remain online-only in the first offline release. A cached explanation is distinguishable from a new request.

Downloads are opt-in and device-specific. Private local data must be cleaned up on logout where feasible and disclosed on shared devices. Browser storage quotas and eviction mean offline availability is not a permanent backup. The reading experience needs a controlled missing-file recovery path.

16.5 Failure and recovery states

An expired session preserves unsent local notes, prompts for authentication and replays only safe idempotent operations after the same account authenticates again. Namespace every local cache, draft and outbox by account. A different account must never see or submit the previous user's local material. Logout or account switching clears or locks that account's local content; define a recovery policy for unsent work before discarding it. An import failure keeps a clear status and lets the user replace the file. A provider outage disables new assistance without blocking reading. A queue outage keeps job intent durable and marks the operation pending. A music error pauses the player with a short explanation.

Preserve the user's selection and draft when retrying. Display real status: saving, saved, saved on this device, syncing, retry needed. Never show success solely because a button was clicked.

17. Verification, operations and cost control

17.1 Verification strategy

Test domain rules where errors could corrupt progress, leak private material or produce misleading statistics. Use unit checks for locators, interval merging, timezone boundaries, streak calculation and permission policies. Use integration checks for private storage access, imports, sessions, idempotency, worker recovery and API contracts. Use end-to-end scenarios for the complete primary reading journeys.

Maintain a permitted fixture set with reflowable EPUB, RTL EPUB, illustrated book, corrupt file, malicious archive fixture, text PDF, scanned PDF and TXT. Scanned PDFs require a separate optional OCR pipeline; do not imply that PDF.js alone supplies selectable text or OCR. Include older document versions to test anchor recovery.

17.2 Release-blocking scenarios

- User B cannot retrieve User A's file, note, image, job result or AI evidence by changing an ID.
- A request cannot include future chapters by fabricating a client locator.
- Duplicate offline events and worker retries do not increase totals or duplicate charges.
- A backward navigation changes current position while preserving distinct reading coverage and spoiler preferences.
- DST changes, midnight, travel and timezone settings do not create ambiguous challenge results.
- An import with active script, unsafe URL or archive traversal cannot execute or escape its limits.
- Audio requires a user start, respects pause and remains independent of reading navigation.
- Failed explanation or image generation has a recoverable visible state.
- Keyboard and screen-reader users can select an action, read the response and return to the book.

17.3 Continuous integration and release

Run type checks, linting, focused automated verification, dependency/security review and production builds. Run API contract compatibility checks and a migration dry run against representative data before schema changes. Keep provider integration checks separate from deterministic tests so transient external failures are diagnosed correctly.

Deploy to staging with sample data. Verify primary journeys, accessibility and backup restoration. Release through a feature flag or a limited cohort. Observe error rate, queue age, missing file errors, AI budget settlement and authorization denials. Roll back application changes independently from irreversible data migrations; use compatible expand-and-contract migrations.

17.4 Observability and support

Use request and job correlation IDs across API, workers and provider adapters. Track structured error codes, p50/p95 latencies, import success, queue lag, retry rates, generation failures, storage growth and per-user quota settlement. Record operational audit events for sensitive actions with actor, target, timestamp and reason.

Support tools should expose state and diagnostics before content. Accessing a user's private passage requires an explicit support workflow and audit record. Alert operators on actionable changes, such as sustained import failures, growing queue delay or spend ceilings, without logging private content.

17.5 Runbooks

Document procedures for provider outage, stuck imports, duplicate job attempts, database restoration, storage failure, leaked credential, abusive account, compromised session, accidental public access and failed deletion. Every runbook identifies the owner, immediate containment, recovery checks and user communication trigger. Test restoration and deletion replay before public launch.

17.6 Cost model without invented prices

Estimate monthly operating cost as infrastructure baseline plus object-storage capacity and requests, data transfer, database usage, queue/cache usage, AI text input/output, image attempts, licensed media and monitoring. Obtain actual vendor quotes at implementation time. No subscription price or vendor rate is assumed in this manual.

For an illustrative workload, 1,000 monthly readers using 20 explanations and 10 translations each create 20,000 explanation requests and 10,000 translations. If 200 readers each request two images, that adds 400 image attempts before retries. Estimate eligible cache hits, context length, output length, peak concurrency and failed attempts separately. This arithmetic is a capacity example, not a revenue forecast or cost quote.

Set a hard operator budget, warnings below it, per-account caps and a transparent user usage view. Keep ordinary reading available when paid generation capacity is exhausted. Avoid an unlimited AI promise until unit economics are measured.

18. Implementation sequence and completion gates

18.1 Milestone 0: product and technical foundation

Confirm launch scope and the default privacy/spoiler policy. Create a design token set, primary user journeys and representative screen prototypes. Evaluate renderer behavior on the fixture set. Select providers and record rights/retention assumptions. Initialize the monorepo, environments, contracts, CI and deployment skeleton.

Completion gate: approved scope, a clickable or navigable reading prototype, verified renderer feasibility, documented unresolved decisions, and a working skeleton build. This milestone establishes the basis for estimating delivery; dates should follow staffing and feasibility work rather than this manual alone.

18.2 Milestone 1: account, private library and import

Implement sessions, account preferences, private uploads, object storage, import jobs and library management. Add format validation, resource limits and truthful processing states. Create immutable document versions and canonical text maps. Add ownership tests before connecting AI.

Completion gate: a user imports a permitted EPUB, sees real metadata and status, can remove it, and another account cannot retrieve its resources. A corrupt or unsafe file fails safely with a useful message.

18.3 Milestone 2: reading quality and annotations

Implement chapters, typography, themes, locations, search, bookmarks, highlights and notes. Persist reading position and activity with idempotent events. Test desktop/mobile, keyboard and RTL cases. Implement distinct current, maximum and AI-boundary state.

Completion gate: the reader resumes accurately across sessions; text selection is stable; annotations reopen in context; layout changes do not inflate progress; duplicate events do not inflate totals.

18.4 Milestone 3: translation and explanation

Connect provider adapters, allowed-context assembly, quotas, caching and error handling. Implement translation and two core explanation modes before expanding the tool menu. Expose evidence references and uncertainty. Evaluate grounding, language quality and spoiler leakage using permitted fixtures.

Completion gate: real translations and explanations work end to end; future chapter evidence is excluded; failures preserve reading; quotas and authorization hold. R1 can ship after the R1 acceptance checklist and accessibility review pass.

18.5 Milestone 4: differentiation and insight

Add scene visualization, Story Memory, Character Compass and vocabulary in small independent increments. Introduce user-started music before adaptive behavior. Add initial recommendations and the private insights screen using recorded events. Instrument usefulness and cost.

Completion gate: image jobs and galleries have correct access; recaps use permitted context; music rights and playback behavior are reviewed; insights explain their metrics; recommendation reasons match actual signals. This is R2.

18.6 Milestone 5: social and offline

Launch friend requests, blocking, explicit sharing and challenge rules. Add discussions only with spoiler controls and moderation. Implement explicit offline downloads and conflict reconciliation after the online data model is stable.

Completion gate: block rules override sharing; a challenge's metric and dates are fixed and explainable; users can read downloaded books and synchronize without duplicates; reports and deletion requests can be handled. This is R3.

18.7 Suggested work ownership

The product owner owns priorities and policy decisions. A designer owns the design system and usability validation. Frontend engineering owns reader interaction and accessibility. Backend engineering owns privacy, ingestion, persistence and API behavior. An AI/integration owner manages providers, evaluation and spend. A QA/release owner coordinates end-to-end validation and rollout. One person may cover several roles in a small team, but each decision still needs a named owner.

19. Acceptance checklist and open decisions

19.1 Product acceptance

- The central reader is usable without music, social features or AI.
- The library, annotations and position use real persisted data.
- Every visible release feature has a successful and failed state.
- Translation, explanations, recaps and scene images respect the configured evidence boundary and disclose model limits.
- Statistics identify native or estimated pages, date range and denominator.
- Recommendations use real signals and can be reset or disabled.
- A private upload never becomes a friend's downloadable book through sharing.
- Reading preferences, privacy, audio and accessibility settings are understandable and reversible.

19.2 Engineering acceptance

- IDs, ownership and document version are verified server-side.
- Client retries, offline queues and background workers are idempotent.
- Queue intents, quotas and expensive provider attempts have auditable state.
- Upload parsing and rendering use explicit limits and restricted execution.
- Backups, restore, deletion and credential rotation have been exercised.
- Performance and accessibility targets are measured on supported devices.
- Provider behavior, rights, regional availability and retention settings are documented accurately.
- There are no fake analytics, placeholder AI outputs or decorative controls presented as complete functionality.

19.3 Decisions still requiring product validation

Decision	Proposed default	Validation needed
Name and brand	Folio, working title	Availability and user response
Initial file format	EPUB	Fixture compatibility and audience demand
Page estimate	250 normalized words per estimated page	Reader comprehension and multilingual fairness
Daily qualifying activity	Five active minutes and non-zero new eligible coverage	Usability and incentive effects
Social profile	Private by default	Comprehension of sharing controls
AI context	Explicit permitted boundary	Read/jump/rewind interaction testing
Music adaptation	Off by default	Distraction, accessibility and rights review
Pricing and quotas	No price set; enforced technical caps	Real provider costs and product testing
Launch languages	English UI, RTL-capable reading; translation set validated	Human language review
Offline availability	Explicit device downloads in R3	Storage reliability and rights

The page estimate and activity threshold are proposed product rules. Native PDF pages, text normalization, timezone policy and challenge metric versions must follow the detailed contracts in Chapter 10 rather than an ambiguous on-screen page number.

20. Detailed workflows and handoff contracts

20.1 Import lifecycle

1. The authenticated user requests an upload slot with filename, expected size, format and checksum. The API checks quota and creates an owned pending asset record.
2. The client uploads to an authorized bounded destination. Completing an upload does not make the content readable immediately.
3. The API verifies stored size/checksum and creates a durable import intent. A worker acquires a lease, validates the archive/document, extracts allowed metadata and quarantines rejected content.
4. The worker creates an immutable edition manifest, sanitized chapters, canonical text map and chunk boundaries. It records the normalization algorithm and version.
5. A final commit marks the edition ready and links it to the user's library. Partial assets remain inaccessible and can be cleaned up after failure.
6. The library shows Ready only after this final state. A retry reuses the logical operation and never creates a second library entry unintentionally.

If an upload is a duplicate, disclose the match only within the user's authorized scope. Never expose whether another private user uploaded the same file. A file replacement creates a new edition and an explicit anchor-migration process.

20.2 Reading, annotation and completion lifecycle

On opening a book, authorize the entry, load its manifest and permitted assets, resolve the saved locator and initialize a local draft queue. Report stable visible ranges rather than a stream of every pixel change. Keep local saves fast, then synchronize batched events and versioned state.

A highlight records an immutable source range, selected quote/fingerprint and chosen color. A note references a source range or a book-level journal entry. Save drafts locally before server submission; merge changes through revision checks. If an anchor cannot be resolved, show an Unplaced note with its preserved excerpt rather than silently moving it to another sentence.

Reaching the final location offers Mark as finished. Track user-declared completion separately from measured coverage. Discovery can use the declared shelf state; a challenge using measured pages uses accepted credits. A new rereading pass is explicit, retains earlier history and never resets the account's historical longest streak.

20.3 Translation/explanation request lifecycle

Resolve the owned edition and selected range, determine the explicit context boundary, validate input length, check quota and look for an eligible cached result. If uncached, reserve budget and assemble bounded evidence. Call the provider through a timeout-controlled adapter, validate the response, persist the output with its configuration/evidence references, settle usage and return or stream the result. On failure, settle or reconcile the reservation and return a typed recoverable state.

Cancellation aborts the client display promptly. The server tracks whether provider processing actually stopped and whether usage occurred. A client disconnect must not leave an unlimited running job or an unaccounted provider attempt.

20.4 Statistics formulas and normalization

For a selected date range, let P be the sum of eligible new unit credits in the range, D the number of included local calendar dates after the account's start, and A the number of dates containing at least one credited unit, regardless of the stricter streak qualification. The default daily average is P / D; an optional active-day average is P / A, shown as unavailable when $A = 0$. Round only the displayed result, preserving the underlying credits.

For EPUB/TXT, proposed normalized units divide canonical text into blocks of 250 words and count the final partial block as one unit. Exclude navigation labels and repeated markup text from the canonical source. Record the segmentation language, Unicode-normalization policy and algorithm version in the edition manifest. Apply a reviewed word-segmentation method for each supported language; do not split only on ASCII spaces and assume this is fair across all writing systems.

Native PDF pages remain independent of zoom. Publisher page labels can be displayed for navigation but do not change the default cross-format scoring rule without a new metric version. Always label combined totals as approximate page units. Eight seconds of accepted unit exposure is an activity threshold, not a claim that 250 words were understood in eight seconds; streak qualification additionally requires five merged active minutes and at least one new qualified unit.

20.5 Challenge scoring example

A private challenge runs from 1-30 November in its published IANA timezone. A participant joins on 3 November, and the rules count only activity after joining. Eligible unit credits are selected by event time in that fixed zone, deduplicated by reading pass/unit, then copied to an auditable contribution record. A late-sync deadline is published before enrollment. The participant score is the sum of accepted contributions, not a client-submitted total.

If a user leaves, remove their name from the active leaderboard and retain only the minimum needed scoring/audit data under the disclosed policy. Edits to a challenge after enrollment cannot silently change its scoring contract; cancel/recreate it or obtain explicit participant agreement to a new revision.

20.6 Proposed configuration inventory

Server-side setting	Purpose	Browser exposure
`APP_ORIGIN`, `API_BASE_PATH`	Public origin and API routing	Only non-secret public equivalents
`DATABASE_URI`	Environment-specific MongoDB connection	Never
`SESSION_SECRET`, session-store settings	Session signing and revocation storage	Never
`OBJECT_STORE_*`	Private bucket and service credentials	Never; signed scoped access only
`QUEUE_CONNECTION`	Worker/queue connectivity	Never
`TEXT_AI_*`, `IMAGE_AI_*`	Provider endpoints, keys and reviewed model config	Never
`UPLOAD_MAX_BYTES`, parser limits	Resource caps	Publish safe UI limits only
`AI_DAILY_LIMITS`, spend ceiling	Per-account and platform cost caps	Publish user-facing allowance only
`METRIC_POLICY_VERSION`	Page, streak and challenge rule version	Include relevant version in responses
`FEATURE_*`	Controlled release availability	Public feature availability, never privileged override

Validate required settings at process startup and fail clearly on missing secrets. Keep secret values out of version control and example files; commit an `.env.example` containing names and safe placeholders only. Health endpoints distinguish basic liveness from dependency readiness and never reveal credentials.

20.7 Design handoff inventory

The design handoff includes semantic tokens; light, sepia and dark reader themes; typography with real mixed-direction samples; component states; desktop/mobile layouts; keyboard and screen-reader behavior; and exact loading/error/empty/sync states. Provide desktop examples near 1440 px and mobile examples near 390 px, then verify intermediate widths and 320 CSS px reflow rather than relying on two screenshots.

Each component specification names its purpose, content limits, variants, interaction, accessible name, keyboard behavior, responsive behavior and source of data. Annotate a book card with fallback-cover rules and long-title wrapping; annotate a metric with its definition and data freshness; annotate an AI output with its machine-generated label, evidence links and failure state.

Use consistent copy: Preparing your book; Saved on this device; Waiting to sync; Explanation unavailable, try again; Estimated pages; Music is off; and Shared with accepted friends. Avoid fake countdowns for provider jobs and guilt-based streak messages. A missing statistic shows Not enough data rather than a fabricated zero.

20.8 Build-review checklist by artifact

Artifact	Reviewer should be able to verify
Product brief	Audience, scope, release boundaries, privacy defaults and open decisions
Design file	Tokens, all primary screens, real states, responsive/RTL/accessibility annotations
OpenAPI contract	Verbs, schemas, auth, ownership, errors, pagination, revisions and idempotency
Data model	Collection ownership, indexes, versioning, retention, deletion and reconciliation
Working reader	Real imports, saved locators, annotation anchors, keyboard/touch and recovery
AI integration	Allowed evidence, provider safety boundaries, evaluation, usage and honest failure
Operational runbooks	Monitoring, budget controls, restore, rollback, deletion and incident response

21. Primary technical references

The following primary sources were checked on 23 October 2026. They support the implementation decisions below; recommendations introduced with "project decision" are proposed architecture, not requirements imposed by a library or standard. Pin and verify supported package releases at implementation time rather than treating this manual as a permanent version inventory.

21.1 React with TypeScript

React documents typed component props, hooks, and React event types. The platform can therefore use TypeScript consistently across reader controls, selection tools, feature panels, and API contracts. **Project decision:** define discriminated unions for EPUB/PDF/TXT locators and explicit states for imports, AI jobs, and playback; keep the reader engine behind a typed adapter so UI components do not depend on EPUB.js internals. Validate incoming requests and provider responses at runtime as a separate application responsibility. React: Using TypeScript

21.2 Express API security

Express recommends TLS, input validation, security headers, secure cookies, protection against repeated failed logins, and maintained dependencies. Apply these to both the main API and upload/import endpoints. **Project decision:** use a server-backed session with secure, HttpOnly cookies; configure SameSite and CSRF protections for the actual deployment topology; validate ownership on each book, annotation, and job. Production session storage must use a persistent supported store rather than the development memory store. Do not return internal error stacks or provider credentials to clients. Express: Production best practices - security

21.3 MongoDB query design

MongoDB explains that indexes reduce scanned documents while adding write overhead, and compound-index field order matters. **Project decision:** begin with indexes that match real access patterns: unique `(userId, editionId)` for a library entry; `(userId, updatedAt)` for recent reading; `(userId, editionId, createdAt)` for annotations; unique `(userId, localDate)` for daily aggregates. Confirm the query plans and latency against representative data before adding more indexes. Keep large book binaries in object storage, with ownership and storage metadata in MongoDB. MongoDB: Indexes, MongoDB: Index types

21.4 EPUB rendering and uploaded-content isolation

EPUB.js supports browser rendering, paginated and scrolling flows, and EPUB navigation. Its own README says embedded scripted content is disabled by default, recommends server sanitization, and warns that enabling `allowScriptedContent` makes the sandbox insecure. **Project decision:** preserve script blocking, reject unsafe URLs and archive paths, enforce decompression limits, and prevent book content from inheriting application credentials. Test fonts, images, links, bidirectional text, and malformed files. A reader library is a rendering component, not a complete upload-security boundary. EPUB.js official repository and README

21.5 Stable EPUB locations and honest page statistics

W3C's EPUB reading-system specification permits dynamic pagination for reflowable publications. EPUB CFI identifies a structural/text location; the specification also recognizes that content edits can invalidate references. **Project decision:** persist CFI plus publication version/hash and a short text-quote fallback. Never use viewport page numbers as permanent annotation addresses. For statistics, show publisher page markers where available; otherwise use explicitly labeled "estimated pages," calculated from a documented, versioned text-unit rule. Reflow after a font-size change must not create new reading credit. W3C: EPUB Reading Systems 3.3, W3C-hosted EPUB CFI specification

21.6 PDF rendering and limits

PDF.js is Mozilla's web-based PDF parsing/rendering platform. Its FAQ describes browser-origin restrictions and practical loading/rendering constraints. **Project decision:** use a patched PDF.js release and worker, render only nearby pages, restrict external actions, and store PDF anchors as document version, page index, text quote, and normalized geometry. Determine whether selectable text exists before offering selection-based AI tools. Scanned pages need a separately scoped OCR process. Mozilla's security advisory history demonstrates why browser rendering must not be described as a guarantee that arbitrary uploaded PDFs are safe. PDF.js home, PDF.js FAQ, Mozilla PDF.js security advisory

21.7 Adaptive music starts with an explicit user action

Chrome documents restrictions on audible autoplay, including Web Audio, and instructs developers to handle rejected playback promises. Other browsers have their own policies. **Project decision:** start every new reading session silent, provide a clear Play control, and start or resume the audio context through that interaction. Show Pause only when playback succeeds. Mood changes may select a suggested track while paused; they must not bypass the user's choice or browser controls. Keep volume, mute, and adaptation controls reachable from the reader. Chrome: Autoplay policy

21.8 Accessibility acceptance target

Use WCAG 2.2 Level AA as the product acceptance target. It includes keyboard access, visible/unobscured focus, contrast, reflow, text-spacing resilience, and minimum pointer-target requirements with specified exceptions. **Project decision:** use 44 CSS-pixel controls where practical, above the standard's 24-pixel minimum; use at least 4.5:1 normal-text contrast and 3:1 large-text contrast; verify all reader themes independently. Translation panels need correct language/direction metadata. Charts need text summaries, illustrations need appropriate alternatives, and music must have direct stop/volume controls. Automated checks support, but do not replace, keyboard and assistive-technology testing. W3C: WCAG 2.2